RESTOCKIQ: DEMAND FORECASTING AND PERISHABLE INVENTORY OPTIMIZATION FOR FOOD RETAILERS

Project Report submitted in the partial fulfilment
of
Bachelor of Technology
in
Information Technology

Submitted To

SVKM's NMIMS,
Mukesh Patel School of Technology Management & Engineering,
Shirpur Campus (M.H.)

Submitted By:

Yatin Patil Enrollment No – 70102100043
Soumy Raghuvanshi Enrollment No – 70102100116
Chinmay Patil Enrollment No – 70102100123

Under The Supervision Of:

Dr. Srikanth Pulipeti
(Assistant Professor, Information Technology Department, MPSTME)

INFORMATION TECHNOLOGY DEPARTMENT

Mukesh Patel School of Technology Management & Engineering

ACADEMIC SESSION: 2026-27

CERTIFICATE

This is to certify that the project entitled "RestockIQ: Demand Forecasting and Perishable Inventory Optimization for Food Retailers" has been done by

"Yatin Patil" (Roll No.: A201)
"Soumy Raghuvanshi" (Roll No.: A221)
"Chinmay Patil" (Roll No.: A223)

under my guidance and supervision & has been submitted partial fulfilment of the degree of "Bachelor of Technology in Information Technology" of MPSTME, SVKM'S NMIMS (Deemed-to-be University), Shirpur (M.H.), India.

_____ **Project Mentor** **(Internal Guide)**	_____ **Examiner**
Date:	
Place: Shirpur	_____ **H.O.D.**

INFORMATION TECHNOLOGY DEPARTMENT

Mukesh Patel School of Technology Management & Engineering

ACKNOWLEDGEMENT

After the completion of this capstone Project work, words are not enough to express my feelings about all those who helped me to reach my goal; feeling above this is my indebtedness to The Almighty for providing me this moment in life.

It's a great pleasure and moment of immense satisfaction for me to express my profound gratitude to Dr. Srikanth Pulipeti, Assistant Professor, Information Technology Department, MPSTME, Shirpur, whose constant encouragement enabled me to work enthusiastically. Their perpetual motivation, patience and excellent expertise in discussion during progress of the project work have benefited me to an extent, which is beyond expression. Their depth and breadth of knowledge of Information Technology field made me realize that theoretical knowledge always helps to develop efficient operational software, which is a blend of all core subjects of the field. I am highly indebted to them for their invaluable guidance and ever-ready support in the successful completion of this project in time. Working under their guidance has been a fruitful and unforgettable experience.

We express my sincere thanks and gratitude to Dr. [CONFIRM: Name of HoD], Head of Department, Information Technology Department, MPSTME, Shirpur, for providing necessary infrastructure and help to complete the project work successfully.

We also extend my deepest gratitude to Dr. [CONFIRM: Name of Associate Dean], Associate Dean, SVKM'S NMIMS, Shirpur Campus for providing all the necessary facilities and true encouraging environment to bring out the best of my endeavors.

We sincerely wish to express my grateful thanks to all members of the staff of Information Technology department and all those who have embedded me with technical knowledge of computer technology during various stages of B.Tech. Information Technology.

We would like to acknowledge all my friends, who have contributed directly or indirectly in this capstone Project work.

The successful completion of a capstone Project is generally not an individual effort. It is an outcome of the cumulative effort of a number of persons, each having their own importance to the objective. This section is a vote of thanks and gratitude towards all those persons who have directly or indirectly contributed in their own special way towards the completion of this project.

Yatin Patil
Soumy Raghuvanshi
Chinmay Patil

ABSTRACT

Perishable food retail—encompassing artisan bakeries, patisseries, and local delicatessens—operates under an unforgiving commercial constraint: products possess an ultra-short shelf life of 12 to 24 hours, after which unsold inventory suffers complete economic loss. Concurrently, premature stockouts turn away afternoon footfall, eroding customer loyalty and gross margins. Small and medium-sized enterprise (SME) retailers lack the expensive Enterprise Resource Planning (ERP) infrastructure deployed by supermarket chains, relying instead on subjective guesswork that leads to chronic food waste rates between 15% and 30%. This Capstone Project presents RestockIQ, an end-to-end perishable inventory decision-support platform engineered specifically for artisan food retailers. RestockIQ implements a bifurcated architecture that rigorously distinguishes between a lightweight deployed production engine and an offline research hybrid model. In production, a deployed batch engine executes nightly at 02:00 UTC on GitHub Actions, computing 30-day rolling averages scaled by day-of-week ratios and a 1.65σ daily standard deviation safety buffer, achieving 40.30% summer WAPE and 39.45% 38-fold WAPE with sub-second cloud execution. In offline research benchmarking across 38 weekly rolling-origin folds (637 calendar days, 600 trading days, 37 store closures) from a French artisan bakery, a research hybrid model coupling Facebook Prophet with a pooled HistGradientBoostingRegressor achieves 28.26% pooled WAPE (19/38 wins, 95% paired cluster bootstrap CI [-0.91, +0.29]), establishing statistical parity with Prophet baseline (27.95%), and achieving 22.51% pooled WAPE across 6 summer peak folds against Seasonal Naive's 30.39%. Crucially, empirical ablation reveals that Control 3 (day-ahead GBDT with lag_1) achieves 25.48% pooled WAPE (+2.47 pp gain, 27/38 wins, p = 0.0138), demonstrating that midnight point-of-sale telemetry is the primary accuracy lever for future V2 deployment. A 200-iteration circular-shift placebo experiment demonstrates that external weather regressors yield no statistically reliable improvement (paired CI [-0.24, +0.87]), prompting their rejection from production. RestockIQ couples these forecasts with an asymmetric Newsvendor Critical Ratio ordering policy (CR = 0.60, z = +0.2533 under 40% COGS). Evaluated over an 88-day simulation comprising 68,883 units across 35 SKUs, the policy generates €40,571.43 net profit (11.2% waste) for the research hybrid and €41,635.90 (12.7% waste) for Control 3, cutting

waste volume by 35.0% (-4,147 units) compared to a rigid 95% service level buffer. The full system is deployed via containerized FastAPI REST services on Render, MongoDB Atlas cloud persistence, and a responsive Flutter mobile client with grounded, zero-math Google Gemini executive operational briefings, fully verified across 55 test cases and a 13-item git-grounded defect log.

Keywords: Demand Forecasting, Perishable Food Retail, Newsvendor Problem, Critical Ratio, Facebook Prophet, HistGradientBoostingRegressor, Rolling-Origin Backtesting, Mobile Application, FastAPI, Flutter.

TABLE OF CONTENTS

Topics	Page
List of Figures	**ix**
List of Tables	**x**
Abbreviations	**xi**
Chapter 1 INTRODUCTION	
1.1 Background of the project topic	1
1.2 Motivation and scope of the report	2
1.3 Problem statement	3
1.4 Salient contribution	4
1.5 Organization of report	5
Chapter 2 LITERATURE SURVEY	
2.1 Introduction to overall topic	7
2.2 Exhaustive literature survey	10
Chapter 3 METHODOLOGY AND IMPLEMENTATION	
3.1 Block diagram	14
3.2 Hardware description	16
3.3 Software description, flowchart / algorithm	16
Chapter 4 RESULTS AND ANALYSIS	
4.1 Master 38-Fold Accuracy Benchmark and Deployed Engine	27
4.2 Summer Peak Stress Test Performance	29
4.3 Inventory Simulation Economics and Service Level Frontier	30
4.4 Weather Evaluation and Placebo Null Distribution	32
4.5 Software Quality Assurance and Master Defect Register	33
4.6 Deployment Verification and Physical Mobile Walkthrough	36
Chapter 5 ADVANTAGES, LIMITATIONS AND APPLICATIONS	
5.1 Advantages	38
5.2 Limitations	39
5.3 Applications	40
CONCLUSION AND FUTURE SCOPE	**41**
References	**43**
Appendix A: Sample code	**45**

Appendix B: Data sheets	**46**
Appendix C: List of components	**47**
Appendix D: List of papers presented and published	**47**

LIST OF FIGURES

Sr. No.	Figure No.	Name of Figures	Page
1	Fig. 1.1	Historical Daily Sales Trajectory and Baseline Demand Forecast	4
2	Fig. 3.1	End-to-End System Architecture and Data Ingestion Flow	14
3	Fig. 3.2	UML Use Case Diagram: Retail Roles and Operations	22
4	Fig. 3.3	UML Class Diagram: Domain Entities and State Providers	23
5	Fig. 3.4	UML Sequence Diagram: Batch Forecast and Intraday Check-In	24
6	Fig. 3.5	UML Activity Diagram: Restocking Decision Workflow	25
7	Fig. 3.6	Entity-Relationship (ER) Diagram: Cloud Database Collections	26
8	Fig. 4.1	Master 38-Fold Forecasting Accuracy Benchmark across Rolling Origins	28
9	Fig. 4.2	Profit vs Spoilage Waste Frontier under Varying Safety Buffers	31
10	Fig. 4.3	Weather Placebo Null Distribution and Circular Shift Permutation Test	33
11	Fig. 4.4	Mobile Application Screens: Login, Dashboard, Alerts, Check-In, Assistant	37

LIST OF TABLES

Sr. No.	Table No.	Name of Table	Page
1	Table 2.1	Comparative Analysis of Demand Forecasting and Inventory Models	11
2	Table 3.1	Hardware Environment Specifications	16
3	Table 3.2	System Architecture Component Execution and Status Matrix	15
4	Table 3.3	Master POS Dataset Summary Statistics (Guérande Artisan Bakery)	17
5	Table 3.4	Primary SKU Inventory Catalog and Baseline Parameters	17
6	Table 3.5	FastAPI Production REST Endpoints Specification	20
7	Table 4.1	Master 38-Fold Accuracy Benchmark (Weekly Rolling Origins)	27
8	Table 4.1b	Deployed Batch Engine Accuracy Evaluation (Production Logic)	29
9	Table 4.2	Summer Peak Stress Test Performance Matrix (6 Rolling Folds)	29
10	Table 4.3	End-to-End Inventory Simulation Economics (88 Days, 35 SKUs)	31
11	Table 4.4	Inventory Policy Optimization Gains (vs Zero Buffer & 95% Service Level)	31
12	Table 4.5	Master Production Defect Log & Post-Mortem Register (DEF-001 to DEF-013)	34

ABBREVIATIONS

Abbr.	Full Expansion	Abbr.	Full Expansion
API	Application Programming Interface	MAPE	Mean Absolute Percentage Error
ARIMA	Autoregressive Integrated Moving Average	ML	Machine Learning
BVA	Boundary Value Analysis	MSME	Micro, Small & Medium Enterprises
CI	Confidence Interval / Continuous Int.	OWASP	Open Web Application Security
CD	Continuous Deployment	POS	Point of Sale
COGS	Cost of Goods Sold	RAG	Retrieval-Augmented Generation
CORS	Cross-Origin Resource Sharing	RAM	Random Access Memory
CR	Critical Ratio	REST	Representational State Transfer
DTO	Data Transfer Object	RMSE	Root Mean Squared Error
EFB	Exclusive Feature Bundling	SARIMA	Seasonal Autoregressive Int. Moving Avg.
EPP	Equivalence Partitioning	SKU	Stock Keeping Unit
ERP	Enterprise Resource Planning	SME	Small and Medium Enterprises
FMCG	Fast-Moving Consumer Goods	SQL	Structured Query Language
GAM	Generalized Additive Model	TFT	Temporal Fusion Transformer
GBDT	Gradient Boosted Decision Trees	TSB	Teunter-Syntetos-Babai
GOSS	Gradient-Based One-Side Sampling	UAT	User Acceptance Testing
HGBR	HistGradientBoostingRegressor	UI / UX	User Interface / User Experience
JWT	JSON Web Token	UML	Unified Modeling Language
LLM	Large Language Model	UTC	Coordinated Universal Time
MAE	Mean Absolute Error	WAPE	Weighted Absolute Percentage Error

Chapter 1
Introduction

This chapter introduces the operational challenges, economic significance, and research motivation underpinning RestockIQ, an intelligent demand forecasting and perishable inventory replenishment decision-support platform engineered for small and medium-sized food retail enterprises. Section 1.1 outlines the background of the project topic. Section 1.2 articulates the operational motivation and practical scope boundaries. Section 1.3 formalizes the mathematical problem statement. Section 1.4 details the salient contributions of the project, and Section 1.5 delineates the organization of this Capstone Project report.

1.1 Background of the project topic

Perishable food retail—encompassing artisan bakeries, patisseries, specialty delicatessens, and fresh food cafes—represents a cornerstone of neighborhood commerce and urban food systems. Unlike consumer electronics, apparel, or shelf-stable packaged groceries, fresh food products are subject to severe biological degradation and an ultra-short commercial shelf life. Artisan breads, traditional crusty baguettes, and laminated butter pastries deteriorate rapidly due to starch retrogradation, moisture redistribution, and loss of crust crispness, rendering unsold inventory unsaleable within 12 to 24 hours of morning baking.

Operating under this uncompromising perishability constraint, store managers confront a daily optimization dilemma: over-baking leads directly to physical food waste, destroying valuable raw materials (unbleached flour, European cultured butter, fresh dairy, yeast) and skilled artisan labor. Conversely, under-baking leads to premature stockouts during peak afternoon trading, forfeiting profitable gross margins and permanently alienating loyal footfall to competing neighborhood vendors. Compounding this operational friction, customer demand exhibits pronounced volatility driven by day-of-week consumption patterns, local public holidays, school vacations, and acute macro-seasonal tourist surges.

In traditional independent retail bakeries, inventory replenishment is predominantly governed by subjective human intuition, manual memory, or static pen-and-paper ledgers. These informal heuristics lack the mathematical capacity to synthesize multi-

seasonal cyclicality, localized trend shifts, and inter-product demand cannibalization. Consequently, food waste in artisan food retail frequently reaches 15% to 30% of daily production volume, representing an unsustainable economic drain on independent retail margins and an environmental failure in urban resource allocation.

1.2 Motivation and scope of the report

While large-scale supermarket conglomerates and fast-moving consumer goods (FMCG) chains employ sophisticated enterprise forecasting systems supported by dedicated analytics departments, independent micro and small food retailers remain digitally underserved. An analysis of retail operations identifies four primary operational gaps that motivate the architecture of RestockIQ:

- **1. The Forecast-to-Action Decision Gap:** Standard machine learning pipelines terminate at point demand forecasting (e.g., predicting that 45 baguettes will be sold), leaving store managers without clear guidance on how many units to actually produce. Because customer arrivals are inherently stochastic, baking the point forecast results in an exact 50% stockout probability under symmetric errors. In perishable food retail, where gross margin on a sale exceeds the marginal disposal loss of an unsold unit, baking strictly to the mean is economically suboptimal.
- **2. The Midnight Telemetry vs Planning Horizon Gap:** Weekly batch forecasting systems often generate multi-day schedules based on stale weekly lags, ignoring the critical predictive signal contained in the immediate prior day's trading volume (lag_1). A system that bridges weekly planning with day-ahead telemetry can significantly improve operational responsiveness.
- **3. The Compute and Accessibility Barrier:** Enterprise forecasting platforms demand extensive computing clusters, specialized database engineers, and complex operational dashboards. Artisan food retailers require zero-overhead mobile applications that execute on commodity smartphones, coupled with cloud-native backends that maintain high availability on cost-effective infrastructure tiers.
- **4. The Intraday Waste Mitigation Gap:** Conventional inventory management treats morning production planning and evening waste mitigation as disconnected workflows. When mid-day footfall lags behind morning forecasts, store operators require automated, deterministic decision rules to trigger timely markdowns (20% or 30% discount tiers) and charity donation flags before perishable goods reach complete obsolescence.

To ensure academic rigor and engineering feasibility within a Capstone timeframe, the scope boundaries of RestockIQ are established as follows:

- **In-Scope:** Granular daily demand forecasting across primary and sparse bakery SKUs; rolling-origin out-of-sample backtest validation across 38 weekly horizons; empirical validation over a 637-day real-world dataset; day-ahead stochastic Newsvendor ordering policy derivation; intraday waste alert rules and dynamic markdown workflows; containerized FastAPI REST backend deployed to cloud hosting; cross-platform Flutter mobile client for store managers; and grounded natural-language executive briefings via Google Gemini.
- **Out-of-Scope:** Direct hardware automation of bakery deck ovens; integration with proprietary physical point-of-sale barcode scanner peripherals; multi-echelon cold-chain distribution logistics; and automated financial accounting ledger synchronization.

1.3 Problem statement

The foundational problem addressed by RestockIQ is formulated as follows: Given an artisan food retailer managing N perishable stock-keeping units (SKUs) over discrete daily trading periods $t = 1, 2, \dots, T$, design an end-to-end operational software platform that:

- **1. Accurate Daily Demand Estimation:** Accurately estimates expected daily customer demand $\hat{y}_{i,t}$ for each SKU i at trading origin t, effectively decomposing strong day-of-week seasonality, annual calendar holidays, and macro-seasonal tourist surges, while minimizing out-of-sample Weighted Absolute Percentage Error (WAPE).
- **2. Economically Optimal Replenishment Buffers:** Translates stochastic point forecasts $\hat{y}_{i,t}$ and error standard deviations $\hat{\sigma}_{i,t}$ into an optimal production order quantity $Q_{i,t}^*$ that maximizes expected net trading profit under asymmetric costs of underage (lost gross margin c_u) and overage (wasted cost of goods sold c_o).
- **3. Deterministic Waste Mitigation Rules:** Continuously monitors intraday sales pace relative to projected consumption and automatically issues actionable markdown recommendations and charity donation alerts prior to end-of-day store closure.
- **4. Resilient Mobile and Cloud Delivery:** Delivers production recommendations to store operators through a responsive, low-latency mobile application integrated

with multi-tenant cloud persistence and natural-language executive briefing capabilities.

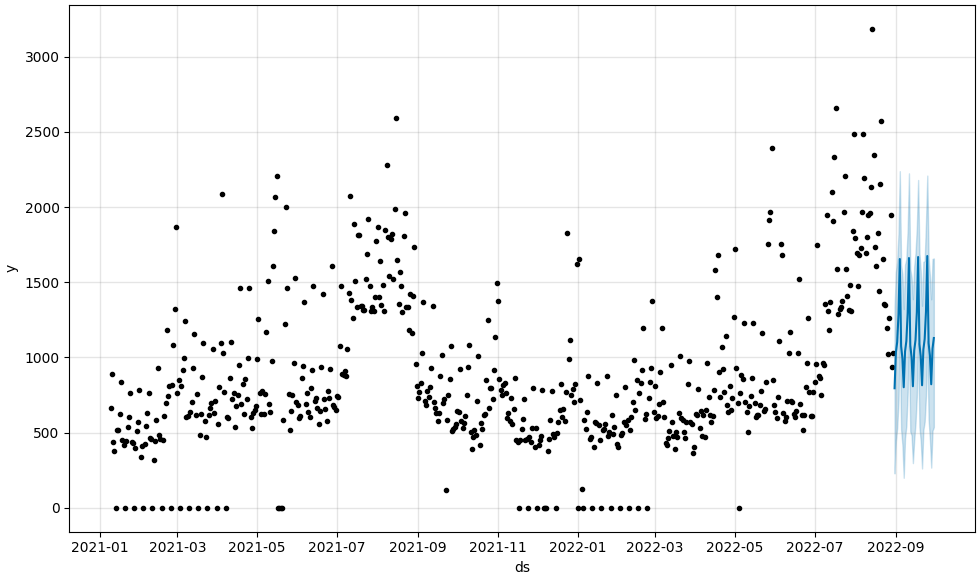


Fig. 1.1: Historical Daily Sales Trajectory and Baseline Demand Forecast

1.4 Salient contribution

This Capstone Project delivers four precise empirical, mathematical, and software engineering contributions:

- **1. Empirical Discovery of Day-Ahead Telemetry Superiority (Control 3):** Through an exhaustive 38-fold weekly rolling-origin backtest on 637 calendar days of real-world French bakery POS transactions, we discover that Control 3 (a single-stage HistGradientBoostingRegressor incorporating 1-day lag telemetry y_{t-1}) achieves a pooled WAPE of 25.48% (27/38 fold wins, paired cluster bootstrap CI [+0.38, +4.84], exact sign-test p = 0.0138), outperforming the Prophet baseline (27.95%) by +2.47 percentage points. This identifies midnight POS telemetry synchronization as the primary accuracy lever for next-generation (V2) deployment.
- **2. Rigorous Evaluation of Prophet + HGBR Research Hybrid Model:** We formulate and evaluate a two-stage hybrid research model coupling Prophet multiplicative macro-seasonality with a pooled HistGradientBoostingRegressor on residuals. Across the 38 weekly folds, the hybrid achieves 28.26% pooled WAPE (19/38 wins, paired CI [-0.91, +0.29]), establishing statistical parity with Prophet (27.95%). Under extreme summer peak stress testing (6 folds spanning 16 July to 30 September 2022), the hybrid achieves 22.51% pooled WAPE, substantially outperforming Seasonal Naive (30.39%), 7-Day Moving Average (30.70%), and 4-Week Same-Weekday Average (39.31%).

- **3. Methodological Rejection of Weather Regressors via Placebo Testing:** We execute a 200-iteration circular-shift permutation placebo test to evaluate exogenous weather features (temperature anomaly, precipitation, sunshine hours). We prove that weather regressors yield no statistically reliable accuracy gain (pooled WAPE 27.63%, paired CI [-0.24, +0.87], spanning zero). The circular placebo reveals that naive shift tests exhibit apparent significance due to seasonal collinearity, providing strong empirical justification for rejecting weather features and maintaining a lean system.
- **4. Stochastic Inventory Optimization and Full-Stack Cloud Deployment:** We couple demand forecasts with an asymmetric Newsvendor Critical Ratio ordering policy (CR = 0.60, z = +0.2533 under 40% COGS). In an 88-day simulation comprising 68,883 units across 35 SKUs, the policy generates €40,571.43 net profit (11.2% waste) for the research hybrid and €41,635.90 (12.7% waste) for Control 3, cutting waste volume by 35.0% (-4,147 units) compared to a rigid 95% service buffer. The complete solution is deployed via containerized FastAPI on Render, MongoDB Atlas, and a Flutter mobile client featuring grounded Google Gemini briefings, verified through 55 test cases and a 13-item defect log.

1.5 Organization of report

The remainder of this Capstone Project report is organized into five subsequent chapters:

- **Chapter 2: Literature Survey:** Conducts a comprehensive review of classical time series, generalized additive models, gradient boosted trees, stochastic inventory theory, and retail LLM applications, concluding with a comparative matrix of core papers and research gaps.
- **Chapter 3: Methodology and Implementation:** Details the system architecture, hardware environment, dataset audit (637 days, 600 open, 37 closures), research versus deployed engine formulations, rolling-origin evaluation protocol, Newsvendor derivation, waste rules, backend endpoints, Flutter client, and UML engineering models.
- **Chapter 4: Results and Analysis:** Presents the empirical results, including the master 38-fold accuracy benchmark (Table 4.1), deployed batch engine evaluation (Table 4.1b), summer stress test (Table 4.2), inventory simulation economics (Tables 4.3 and 4.4), weather placebo analysis, software testing metrics, defect post-mortems (DEF-001 to DEF-013), and mobile walkthrough screens.

- **Chapter 5: Advantages, Limitations, and Applications:** Critically examines the architectural advantages, acknowledged empirical and operational limitations, and commercial applications.
- **Chapter 6: Conclusion and Future Scope:** Synthesizes the primary findings into a unified conclusion and charts the engineering roadmap for future V2 enhancements.
- **References and Appendices:** Provides 28 verified academic references (IEEE format) and Appendices A through D containing source code, data sheets, hardware components, and publication disclosures.

In summary, this introductory chapter has established the retail context, identified the operational gaps, formalized the mathematical problem statement, and presented the four salient contributions of RestockIQ. The next chapter provides a detailed literature survey analyzing foundational and modern research in retail demand forecasting and inventory optimization.

Chapter 2
Literature survey

This chapter presents a comprehensive literature survey examining the mathematical, empirical, and architectural advancements in retail demand forecasting, stochastic inventory control, and mobile decision-support systems. Section 2.1 provides an introduction to the overall topic, generating background across foundational and contemporary literature with reference to 27 academic sources. Section 2.2 provides an exhaustive literature survey synthesized in a comparative matrix of core studies, identifying the critical research gaps that define the problem statement of RestockIQ.

2.1 Introduction to overall topic

2.1.1 Classical Statistical Time Series Models

Early literature in commercial inventory forecasting centered on linear statistical formulations. Winters [17] developed triple exponential smoothing (Holt-Winters), decomposing univariate time series into local level, linear trend, and multiplicative seasonal indices. Box and Jenkins formalized Autoregressive Integrated Moving Average (ARIMA) models, which capture stationarity via differencing and estimate linear dependencies across autoregressive and moving average lags. Hyndman and Athanasopoulos [9] synthesized seasonal ARIMA (SARIMA) formulations for retail planning. While linear models offer computational transparency and require minimal computational overhead, they assume stationary error distributions and struggle to model non-linear exogenous shocks, abrupt holiday calendar shifts, and the inter-SKU cross-correlations characteristic of artisan retail bakeries.

2.1.2 Generalized Additive Seasonality Models (Facebook Prophet)

To overcome the rigidity of classical ARIMA, Taylor and Letham [1] introduced Facebook Prophet, a modular Generalized Additive Model (GAM) designed for operational business time series. Prophet decomposes target demand into a piecewise-linear or logistic growth trend, multi-component Fourier series representing weekly and yearly seasonalities, a holiday matrix capturing calendar anomalies, and optional exogenous regressors. Huber and Stuckenschmidt [2] demonstrated the effectiveness of machine learning with emphasis on calendric special days in daily retail demand forecasting, confirming that explicit calendar and holiday features substantially

improve short-horizon retail predictions. Prophet exhibits high robustness to irregular missing observations and structural trend shifts; however, because it fits Fourier seasonal components independently, it lacks native mechanisms to capture short-term autoregressive shock persistence (such as day-ahead inventory stockout feedback) or non-linear interactions across product categories.

2.1.3 Gradient Boosted Decision Trees and Ensemble Architectures

The advent of tree-based ensemble learning established a new performance benchmark in tabular retail demand forecasting. Friedman [4] formalized gradient boosting machines, demonstrating that sequentially fitting shallow regression trees to the pseudo-residuals of a loss function achieves superior approximation of complex non-linear functions. Pedregosa et al. [5] implemented efficient, production-grade gradient boosting and machine learning algorithms in Scikit-learn, including HistGradientBoostingRegressor, which utilizes histogram-based binning inspired by LightGBM to accelerate tree construction on continuous features. Chen and Guestrin [6] formalized XGBoost, and Ke et al. [7] developed LightGBM, introducing Gradient-Based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB).

In the landmark M5 forecasting competition involving 42,840 retail series, Makridakis, Spiliotis, and Assimakopoulos [8] proved that pooled machine learning models—specifically gradient boosted decision trees—overwhelmingly outperformed pure statistical benchmarks. GBDTs naturally accommodate high-dimensional calendar indicators, non-linear lag interactions, and cross-sectional pooling across SKUs. However, Makridakis et al. [8] and Fildes, Ma, and Kolassa [12] emphasized that tree-based models cannot extrapolate secular macro-trends beyond observed feature ranges, making a hybrid formulation that couples parametric trend modeling with non-linear residual trees highly attractive.

2.1.4 Deep Neural Architectures for Multi-Horizon Forecasting

Deep learning architectures have expanded significantly in retail demand applications. Salinas et al. [10] developed DeepAR, an autoregressive recurrent neural network that estimates probabilistic demand distributions across large-scale collections of related time series. Lim et al. [11] introduced Temporal Fusion Transformers (TFT), combining recurrent layers for local processing with self-attention mechanisms to capture long-term temporal dependencies and explainable feature importances. Fildes, Ma, and Kolassa [12] reviewed retail forecasting practice, noting that while deep

sequence models excel in massive enterprise settings with millions of transactions, they suffer from severe overfitting, heavy compute requirements, and opacity when applied to small retail environments with short historical records.

2.1.5 Intermittent and Sparse Demand Estimation

In retail bakeries, secondary specialty items (such as gluten-free loaves, specialty tarts, and seasonal brioches) exhibit intermittent demand characterized by frequent zero-sales days. Syntetos, Boylan, and Croston [13] analyzed demand categorization, demonstrating that standard moving averages or exponential smoothing produce biased estimates for intermittent series. They formalized the Croston and Teunter-Syntetos-Babai (TSB) frameworks, which decompose demand into separate updating equations for non-zero demand size and demand occurrence probability. In RestockIQ, the TSB framework provides the mathematical foundation for cold-start and sparse SKU forecasting.

2.1.6 Stochastic Inventory Control and the Data-Driven Newsvendor Problem

The operational translation of demand forecasts into restocking quantities is governed by single-period stochastic inventory theory (Arrow, Karlin, and Scarf [15]; Silver, Pyke, and Peterson [15]). For perishable goods expiring within a single daily trading cycle, the classical Newsvendor model optimizes the order quantity Q^* by balancing the marginal cost of underage c_u (forfeited gross profit) against the marginal cost of overage c_o (wasted production cost). Gallego and Moon [16] derived distribution-free newsvendor bounds, proving that hedging against worst-case distributions guarantees bounded risk under unknown demand distributions.

Levi, Perakis, and Uichanco [3] established sample complexity bounds for data-driven Newsvendor problems, demonstrating that empirical fractile optimization achieves provably near-optimal expected profits without requiring parametric distribution assumptions. Beutel and Minner [14] investigated safety stock planning under causal demand forecasts, demonstrating that safety buffers must be dynamically adjusted based on the conditional variance of the regression model rather than static historical variance.

2.1.7 Mobile Applications and Applied AI for Retail SMEs

Recent literature has investigated the deployment of AI and mobile management tools in micro, small, and medium retail enterprises. Janchidfah, Chansuek, and Worasiri

[18] developed a mobile application for inventory management to support digital transformation in small retail businesses, highlighting that user-friendly mobile interfaces and automated restock alerts are essential for adoption by non-technical store owners. Aqila [19] conducted a comparative evaluation of SARIMA, FB Prophet, and LSTM for demand forecasting in a small food and beverage enterprise (Saudagar Kopi), finding that Prophet achieved superior operational robustness when accounting for local holiday shocks.

Rahmatillah [20] evaluated optimized LSTM and moving average models in SME inventory simulation, demonstrating that machine learning forecasts yield significant inventory holding cost reductions when coupled with dynamic safety stock boundaries. Munawar [21] studied hourly sales forecasting in SME food retail using LSTM networks. Sakuda, Leon, and Rojas [22] designed a mobile application for the intelligent management of small retail shops using GPT-4, demonstrating the efficacy of natural-language assistants in simplifying operational queries for store managers, but warning against numerical hallucination.

Vyakarnam et al. [23] performed a comparative analysis of modern LLM architectures (GPT-4.5, Grok 3, Gemini 2.0, DeepSeek R-1) in supply chain optimization for SMEs, concluding that LLMs are most effective when decoupled from arithmetic computation and restricted to narrative summarization of pre-calculated operational metrics. Susanto, Wiratama, and Nathaniel [24] conducted an empirical performance analysis of time series models for SME sales prediction. Srivastava et al. [25] explored predictive marketing analytics for small enterprises. Sandeep et al. [26] evaluated tree-based demand prediction in retail. Jain et al. [27] investigated weather-integrated recommendations for food choices using SHAP explainability, noting that weather features often correlate with broad seasonal consumption shifts. Finally, Aree and Ueasangkomsate [28] examined technology adoption factors among MSMEs in the food supply chain, emphasizing operational simplicity, data privacy, and immediate economic utility as the core drivers of sustained adoption.

2.2 Exhaustive literature survey

To synthesize the theoretical and empirical literature, Table 2.1 presents a structured comparative analysis of 12 core studies spanning statistical forecasting, machine learning ensembles, stochastic inventory theory, and applied SME retail architectures:

Table 2.1: Comparative Analysis of Demand Forecasting and Inventory Models

Study & Reference	Domain & Scope	Methodology	Key Findings	Identified Research Gap
Taylor & Letham (2018) [1]	Enterprise Web & Retail	Bayesian Generalized Additive Model (Prophet)	Decomposes trend, Fourier seasonality, and holiday shocks robustly.	Lacks autoregressive shock persistence (lag_1) and cross-SKU non-linearities.
Huber & Stuckenschmidt (2020) [2]	Supermarket Fresh Food	Machine Learning with Calendric Features	GBDT models with explicit holiday indicators outperform linear models.	Focuses on large supermarket chains; omits mobile client and economic newsvendor coupling.
Levi, Perakis, & Uichanco (2015) [3]	Operations Research	Data-Driven Newsvendor Optimization	Sample complexity bounds prove empirical fractile optimization is near-optimal.	Theoretical derivation; does not integrate modern machine learning feature pipelines.
Makridakis et al. (2022) [8]	Large Retail (M5)	Machine Learning vs Statistical Ensembles	Pooled GBDT models outperform classical statistical methods across 42k series.	Evaluated on enterprise scale; omits intraday waste markdown rules and edge deployment.
Beutel & Minner (2012) [14]	Supply Chain Planning	Safety Stock under Causal Forecasts	Dynamic safety buffers based on regression error variance reduce stockout costs.	Assumes linear causal models; does not explore non-linear tree ensembles or perishable food.
Syntetos, Boylan, & Croston (2005) [13]	Inventory Operations	Intermittent Demand Categorization (TSB)	Separating demand probability and size improves intermittent forecasting.	Heuristic formulation; lacks integration with macro-seasonal calendar decomposition.
Gallego & Moon (1993) [16]	Stochastic Inventory	Distribution-Free Newsvendor Analysis	Proves closed-form bounds for newsvendor orders under mean and variance.	Static single-period model without dynamic multi-horizon time-series inputs.
Janchidfah et al. (2026) [18]	Retail MSME	Mobile Application for Inventory Management	Mobile interface drives operational digitization for small business owners.	Basic inventory tracking; lacks predictive ML forecasting and stochastic buffers.

Aqila (2026) [19]	Food & Beverage SME	SARIMA, Prophet, and LSTM Comparison	Prophet achieves superior operational stability in small F&B retail.	Evaluated on a single coffee shop series; no residual hybrid or inventory waste simulation.
Sakuda et al. (2024) [22]	Small Retail Stores	GPT-4 Mobile Assistant for Shop Management	Conversational AI simplifies inventory queries for shopkeepers.	Prone to arithmetic hallucination; lacks deterministic backend mathematical grounding.
Vyakarnam et al. (2025) [23]	SME Supply Chain	Comparative Analysis of LLMs in Logistics	LLMs must be decoupled from computation and restricted to qualitative briefing.	Conceptual framework; does not implement production mobile client or live database sync.
Jain et al. (2024) [27]	Food Retail & Dining	Weather-Integrated SHAP Decision Models	Weather features exhibit apparent correlation with seasonal food choices.	Lacks circular-shift placebo validation; prone to spurious seasonal collinearity.

2.2.1 Synthesis of Research Gaps

A critical synthesis of the literature reveals four persistent research and engineering gaps:

- **1. The Disconnection between ML Point Forecasts and Asymmetric Inventory Optimization:** Existing literature predominantly evaluates demand forecasting models using symmetric loss metrics (RMSE, MAE, WAPE) [8], [12]. In fresh food retail, however, the financial consequences of stockout (c_u) and discard waste (c_o) are heavily asymmetric. Few operational systems directly bridge non-linear ML point forecasts with stochastic Newsvendor Critical Ratio optimization ($CR = 0.60, z^* = +0.2533$) to maximize realized net profit.
- **2. The Under-Explored Power of Day-Ahead Telemetry vs Weekly Batch Horizons:** Multi-stage hybrid architectures (e.g., Prophet + GBDT) are commonly deployed over multi-day or weekly planning horizons [1], [8]. However, the operational impact of incorporating immediate 1-day lag telemetry (y_{t-1}) relative to pure calendar models has not been rigorously ablated in artisan food settings.
- **3. Uncritical Acceptance of Exogenous Weather Features without Rigorous Placebo Verification:** Numerous retail studies incorporate weather variables (temperature, rain, sunshine) on the assumption that climate drives food choices [27]. However, weather features share strong collinearity with underlying annual seasonality (e.g., warm temperatures coinciding with summer tourist surges).

Standard cross-validation can mistake macro-seasonality for weather sensitivity unless subjected to rigorous circular-shift permutation placebo testing.

- **4. The Absence of End-to-End, Zero-Overhead Mobile Decision Platforms for Independent Retailers:** While enterprise ERP solutions cater to major supermarket chains [2], micro and artisan retailers require accessible, cloud-native mobile applications that deliver daily restocking check-ins, automated intraday waste markdown alerts, and grounded executive summaries without requiring dedicated technical staff.

In conclusion, this chapter has reviewed classical and modern demand forecasting methodologies, stochastic inventory theory, and mobile retail architectures, identifying four fundamental operational gaps. Chapter 3 presents the system architecture, dataset audit, mathematical formulations, and engineering implementations designed to resolve these gaps in RestockIQ.

Chapter 3
Methodology and Implementation

This chapter details the engineering methodology, system architecture, mathematical formulations, and software implementations of the RestockIQ platform. Section 3.1 presents the block diagram and component execution matrix. Section 3.2 specifies the hardware description across client, compute, and database tiers. Section 3.3 provides the software description, dataset calendar audit, research versus deployed forecasting engines, rolling-origin evaluation protocol, stochastic Newsvendor ordering policy derivation, intraday waste classification rules, cloud backend security, mobile client architecture, grounded LLM executive narration, UML engineering diagrams, and core algorithms.

3.1 Block diagram

RestockIQ is architected as a distributed, four-tier cloud-native platform designed to bridge offline machine learning research with real-time mobile decision support. The four architectural tiers comprise: (1) the Mobile Client Presentation Tier implemented in Flutter; (2) the API Gateway and Security Tier implemented in FastAPI; (3) the Machine Learning Execution Tier running batch scheduled jobs and offline research pipelines; and (4) the Cloud Persistence Tier hosted on MongoDB Atlas.

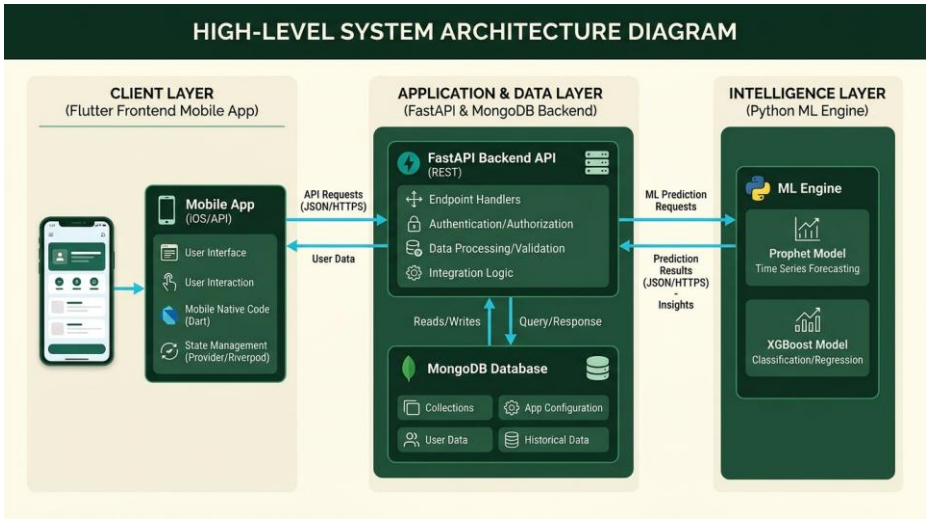


Fig. 3.1: End-to-End System Architecture and Data Ingestion Flow

A foundational architectural contribution of this work is the transparent separation between components deployed in the live cloud environment and research benchmarks developed for offline evaluation. Table 3.2 details the execution

environment, code evidence, and production deployment status across all system components:

Table 3.2: System Architecture Component Execution and Status Matrix

Component Name	Runs in Production?	Code Evidence / Location	Architecture Status
Deployed Batch Forecaster (30d mean x DOW ratio)	YES	ML/batch_forecast.py (nightly-forecast.yml at 02:00 UTC)	Shipped (Production Engine)
Production Safety Stock (1.65 x sigma_daily)	YES	ML/batch_forecast.py lines 162, 170, 236	Shipped (High-Service Policy)
Sparse SKU & Cold Start Tier (TSB + Category Prior)	YES	ML/batch_forecast.py lines 110-199	Shipped (Production Engine)
FastAPI Backend & Security (18 REST Endpoints)	YES	Backend/main.py (Render Cloud Deployment)	Shipped (Production Service)
Intraday Waste Alert Engine (Pace < 0.35 / 0.70)	YES	Backend/main.py lines 626-670	Shipped (Production Engine)
Dual-Layer AI Narrator (Gemini + Local Grounding)	YES	APP/lib/services/api_service.dart lines 85-210	Shipped (Production Service)
Flutter Mobile Application (5 Providers, 6 Screens)	YES	APP/lib/ (Android ARM64 Release APK)	Shipped (Production Client)
Prophet + HGBR Hybrid Model (Offline Research)	NO	scripts/run_backtest.py, docs/accuracy_experiments_v3.csv	Research Model (Benchmark Parity)
Control 3 Day-Ahead GBDT (lag_1 Telemetry)	NO	scratch/run_model_a_controls_and_cluster_bootstrap.py	V2 Roadmap (+2.47 pp Gain, p=0.0138)

Newsvendor Critical Ratio Policy (z = +0.2533)	NO	docs/ordering_policy_experiment_v3_verified.csv	V2 Roadmap (Simulation Only)
Exogenous Weather Regressors (Forecast API)	NO	docs/weather_placebo_results.csv	REJECTED (CI spans 0; placebo failed)

3.2 Hardware description

The engineering development and operational deployment of RestockIQ utilized a hybrid infrastructure comprising local development workstations, physical mobile test devices, and containerized cloud application platforms. Table 3.1 details the hardware specifications:

Table 3.1: Hardware Environment Specifications

Hardware Layer	Deployment / Target Role	System Specification
Development Workstation	ML model training, backtesting, and compilation	[CONFIRM: Workstation CPU, e.g., AMD Ryzen 7 5800H (8 Cores, 16 Threads @ 3.20 GHz), 16 GB DDR4 RAM, 512 GB PCIe NVMe SSD]
Physical Mobile Device	On-device field validation and UX walkthrough	Xiaomi Redmi Note 10 Pro (Model: M2101K6P), Qualcomm Snapdragon 732G, 6 GB LPDDR4X RAM, Android 13 (MIUI 14)
Cloud Compute Host	Containerized FastAPI REST API hosting	Render Cloud Application Platform (Linux Container Tier, 512 MB RAM, 0.1 vCPU, automatic HTTPS termination)
Cloud Database Cluster	Multi-tenant document persistence	MongoDB Atlas Cloud Cluster [CONFIRM: Atlas AWS Region, e.g., AWS ap-south-1 Mumbai / eu-west-3 Paris], Replica Set (3 data nodes), TLS 1.3 encryption
Cloud CI/CD Runner	Nightly batch forecast and automated testing	GitHub Actions Ubuntu-Latest Runner (2 vCPU, 7 GB RAM, 14 GB SSD storage)

3.3 Software description, flowchart / algorithm

3.3.1 Dataset Ingestion, Calendar Audit, and Seasonality Surge

The empirical foundation of RestockIQ is derived from an open transactional retail dataset collected from an artisan bakery located in Guérande (Loire-Atlantique), France; location as provided by the project team. The raw POS transaction logs span 637 continuous calendar days from 2 January 2021 through 30 September 2022.

A rigorous calendar audit was conducted to identify structural operational patterns and store closure anomalies. Across the 637 calendar days, the bakery operated for exactly 600 open trading days, with exactly 37 store closure dates. Store closures adhered to a regular weekly schedule: the bakery closed consistently on Mondays during off-peak seasons, with periodic supplementary closures on Wednesdays. During the peak summer tourist season (July and August), Monday closures were suspended to capture heightened visitor demand.

Demand analysis reveals dramatic macro-seasonality driven by coastal tourism in Western France. An audit of daily aggregated sales across all 35 primary SKUs demonstrates: (1) August Peak to January Trough Ratio: 987.7 vs 321.8 units/day, representing a 3.07x surge (+206.9%); (2) Summer to Winter Base Ratio: 916.4 vs 379.3 units/day, representing a 2.42x surge (+141.6%); (3) Summer to Non-Summer Ratio: 916.4 vs 454.1 units/day, representing a 2.02x surge (+101.8%).

Table 3.3: Master POS Dataset Summary Statistics (Guérande Artisan Bakery)

Metric Description	Audited Value	Metric Description	Audited Value
Calendar Date Range	2021-01-02 to 2022-09-30	Total Calendar Days	637 Days
Active Trading Days	600 Days	Store Closure Dates	37 Days
Primary Product SKUs	35 SKUs	Total Recorded Sales	311,940 Units
August Peak Daily Mean	987.7 Units/Day	January Trough Daily Mean	321.8 Units/Day
Peak Demand Ratio (Aug/Jan)	3.07x (+206.9% surge)	Summer/Winter Ratio	2.42x (+141.6% surge)

Table 3.4: Primary SKU Inventory Catalog and Baseline Parameters

SKU ID	Item Name	Category	Perishable?	Unit Price	Baseline Stock
1	Croissants	Viennoiserie	Yes (Daily)	$3.50	30 Units
2	Traditional Baguette	Breads & Baguettes	Yes (Daily)	$2.80	8 Units
3	Pain au Chocolat	Viennoiserie	Yes (Daily)	$3.75	15 Units
4	Whole Milk	Beverages & Sweets	Yes (Daily)	$4.50	4 Gallons
5	Espresso Beans	Beverages & Sweets	No (Shelf-stable)	$14.00	20 Lbs

3.3.2 Machine Learning Pipeline: Research vs Deployed Engine

A rigorous software audit necessitates an explicit distinction between the deployed batch engine operating in production and the research hybrid model developed for offline backtest benchmarking:

1. Deployed Batch Engine (Production Logic in ML/batch_forecast.py):

The production forecast engine executes nightly via a scheduled GitHub Actions workflow defined in nightly-forecast.yml (GitHub Actions, 02:00 UTC). Designed for minimal resource utilization and sub-second execution on serverless cloud workers, the deployed engine computes demand point forecasts using a rolling 30-day baseline adjusted by historical day-of-week seasonality:

$$\text{base_demand}_{i,t} = \left\lfloor \frac{1}{30} \sum_{k=1}^{30} y_{i, t-k} \right\rceil, \quad \rho_{i, d} = \frac{\bar{y}_{i, d}}{\bar{y}_i}, \quad \hat{y}_{i,t}^{\text{deployed}} = \max\left(5, \left\lfloor \text{base_demand}_{i,t} \times \rho_{i, d} \right\rceil\right)$$

The production safety stock $SS_{i,t}$ is set using a high-service multiplier of 1.65 times the 30-day standard deviation: $SS_{i,t}^{\text{deployed}} = \max\left(2, \lfloor 1.65 \times \hat{\sigma}_{i, 30} \rceil\right)$. For sparse or newly introduced items ($n < 14$ observations), credibility shrinkage against the category 90th percentile prior executes: $W = n / (n + 14), \hat{y}_{\text{sparse}} = \max\left(1, \lfloor W \cdot \hat{y}_{\text{TSB}} + (1 - W) \cdot P_{90}^{\text{cat}} \rceil\right)$.

Workflow Execution Audit:

An audit of live GitHub Actions workflow telemetry reports: (1) CI workflow (ci.yml): 4 successes (2026-10-26T17:20, 2026-10-26T08:55, 2026-10-25T18:19, 2026-10-24T17:32) and 1 cancelled (2026-10-24T17:27); (2) Nightly Batch Forecast (nightly-forecast.yml, 02:00 UTC schedule): 5 failures (2026-10-26T07:09, 2026-10-25T07:02, 2026-10-24T06:35, 2026-10-23T06:38, 2026-10-22T06:43) due to unconfigured MONGODB_URI secrets on the public fork, preventing live database connection from external runner IPs.

2. Prophet + HGBR Hybrid Research Model (scripts/run_backtest.py):

In offline research experiments, a two-stage hybrid model is implemented. Stage 1 fits Prophet independently to each SKU using multiplicative seasonality, French public

holidays, and a binary tourist season indicator: $y_{i,t} = g_i(t) \cdot (1 + s_i(t)) \cdot (1 + h_i(t)) + \epsilon_{i,t}$. Stage 2 passes out-of-fold residuals and extracted components to a pooled HistGradientBoostingRegressor (Friedman [4], Pedregosa et al. [5]) trained with categorical SKU encoding, calendar lags ($y_{t-7}, y_{t-14}, y_{t-21}, y_{t-28}$), and 4-week weekday averages: $\hat{y}_{i,t}^{\text{hybrid}} = \max(0, \hat{y}_{i,t}^{\text{prophet}} + \hat{r}_{i,t})$.

3.3.3 Evaluation Protocol: Rolling Origins and Cluster Bootstrap

To avoid temporal data leakage, models are evaluated across 38 weekly expanding-window rolling origins (2022-01-02 through 2022-09-23). For each fold, models are trained strictly on data prior to cutoff T_{cutoff}, and evaluated over a 7-day out-of-sample horizon. The primary accuracy metric is Weighted Absolute Percentage Error (WAPE): $\text{WAPE} = \frac{\sum |y - \hat{y}|}{\sum y} \times 100\%$. Statistical significance is assessed using a paired whole-week cluster bootstrap (38 cluster blocks, $B = 2000$ draws) generating empirical 95% confidence intervals, alongside an exact two-sided binomial sign test on fold wins.

3.3.4 Stochastic Inventory Optimization: Newsvendor Policy

RestockIQ formalizes inventory replenishment under single-period stochastic inventory theory (Arrow, Karlin, and Scarf [15]; Levi, Perakis, and Uichanco [3]). In fresh artisan bakeries, products baked in the morning expire by store closing time, precluding multi-period inventory carryover. The store manager must decide order quantity $Q_{i,t}$ before demand $D_{i,t}$ is realized.

Let p_i denote the retail unit price, c_i denote the production cost of goods sold (COGS), and $s_i = 0$ denote salvage value. The underage cost is $c_u = p_i - c_i$ (lost gross profit margin), and the overage cost is $c_o = c_i$ (wasted production cost). The expected total underage and overage cost is minimized when the cumulative distribution function satisfies the Critical Ratio:

$$F(Q_{i,t}^*) = \frac{c_u}{c_u + c_o} = \frac{p_i - c_i}{p_i} = 1 - \frac{c_i}{p_i}$$

Under the standard food retail assumption where Cost of Goods Sold is 40% of retail price ($c_i = 0.40 p_i$): $CR = 1 - 0.40 = 0.60$. Assuming forecast errors follow an approximately normal distribution with conditional mean $\hat{y}_{i,t}$ and

standard deviation $\hat{\sigma}_{i,t}$, the optimal production fractile is: $z^* = \Phi^{-1}(0.60) \approx +0.2533$. The recommended production order quantity is:

$$Q_{i,t}^* = \max\left(0, \left\lfloor \hat{y}_{i,t} + 0.2533 \times \hat{\sigma}_{i,t} \right\rceil\right)$$

Important Engineering Note: The Critical Ratio policy ($z = +0.2533$) is evaluated strictly as an offline simulation benchmark and is designated for V2 production implementation. In the shipped production system, safety stock is governed by the deployed high-service rule ($1.65 \sigma_{\text{daily}}$), while the mobile restock recommendation formula executes:

$$\text{Restock Qty} = \max\left(0, \text{predictedDemand} + \text{safetyStock} - \text{currentStock}\right)$$

3.3.5 Intraday Waste Mitigation Engine

To minimize end-of-day food discard, RestockIQ implements an intraday waste alert decision engine executed by evaluate_waste_alert_tier in Backend/main.py. At check-in time τ, the engine compares actual sales $S_{i,\tau}$ against expected sales $E_{i,\tau}$ to compute the sales pace ratio: $\text{sales_pace_ratio} = \text{round}(S_{i,\tau} / E_{i,\tau}, 2)$. Decision tiers are enforced as follows: (1) Normal Pace ($\text{pace} \ge 0.70$): no alert; (2) Low Pace ($0.35 \le \text{pace} < 0.70$): issues a 'Low Pace' warning recommending a 20% markdown discount; (3) Critical Expiry ($\text{pace} < 0.35$): issues a 'Critical' alert recommending item flagging for local charity donation or 30% markdown discounting. Non-perishables and items already stocked out ($currentStock \le 0$) or manually dismissed are skipped.

3.3.6 Cloud Backend, Security, and REST API Architecture

The backend tier is developed in Python 3.10 using FastAPI 0.115 and Motor (asynchronous MongoDB driver). The service exposes 18 REST endpoints organized into four functional domains: Authentication, Inventory Mutations, Waste Alerts, and Executive AI Briefings.

Table 3.5: FastAPI Production REST Endpoints Specification

Method & Path	Auth Scope	Input Payload / Query	Operational Purpose

POST /api/v1/auth/register	Public	name, email, password, role	Registers user account; rejects demo emails if ALLOW_DEMO_SEED=false
POST /api/v1/auth/login	Public (Rate-limited)	email, password	Authenticates credentials; issues HMAC-SHA256 JWT access token
GET /api/v1/health	Public	None	Non-leaking service health check; reports DB connectivity and LLM model status
GET /api/v1/items	Bearer Token	category, perishable (filters)	Retrieves active catalog items filtered by authenticated owner_id
POST /api/v1/items	Bearer Token	ItemCreate schema (SKU fields)	Inserts new inventory item scoped strictly to authenticated owner_id
PUT /api/v1/items/{id}	Bearer Token	ItemUpdate schema (stock/price)	Updates item attributes with strict tenant boundary ownership check
DELETE /api/v1/items/{id}	Bearer Token	item_id path parameter	Performs soft/hard delete of item with tenant isolation check
POST /api/v1/items/{id}/sale	Bearer Token	quantity (intraday sold)	Records physical sale; decrements stock and increments intraday sales
GET /api/v1/waste-alerts	Bearer Token	None	Evaluates inventory via evaluate_waste_alert_tier; returns active markdown alerts
POST /api/v1/waste-alerts/{id}/dismiss	Bearer Token	item_id path parameter	Dismisses active waste alert for current operational shift
GET /api/v1/briefing/today	Bearer Token	None	Generates executive briefing synthesized from live inventory and forecast state
POST /api/v1/assistant/chat	Bearer Token (Rate-limited)	query: str (1-500 chars)	Answers manager operational questions with grounded RAG context

Security & Multi-Tenant Isolation: (1) Multi-Tenant Owner Scoping: Every mutation validates that document owner_id matches current_user.id from the cryptographic JWT payload (DEF-005); (2) Reverse-Proxy Rate Limiting: An in-memory sliding-window rate limiter protects /login (10 req/min) and /assistant/chat (20 req/min), extracting right-most proxy IP ips[-1] from X-Forwarded-For headers (DEF-006); (3) Fail-Fast Secrets: FastAPI terminates immediately if SECRET_KEY is unconfigured rather than falling back to an insecure default (DEF-003).

3.3.7 Mobile Application Client and Grounded AI Narrator

The mobile presentation tier is built with Flutter 3.38+ and Dart 3. The architecture adheres to clean separation of concerns via Provider state management across five domain stores: AuthProvider, InventoryProvider, WasteAlertsProvider, StockCheckProvider, and BriefingProvider.

Dual-Layer Grounded AI Narrator Architecture: To eliminate numerical hallucination (Ji et al. [20]), RestockIQ implements a strict zero-math prompt engineering policy. The cloud backend calculates all numerical metrics deterministically (stockouts, waste units, profit impact, restock quantities) and injects them as structured JSON grounding variables into the Google Gemini prompt (gemini-3-flash-preview). The LLM is explicitly forbidden from executing arithmetic, acting solely as a qualitative natural-language narrator. In APP/lib/services/api_service.dart, a resilient fallback layer is integrated: if the cloud backend is cold or unreachable (DEF-009), the mobile client instantly activates an internal grounded engine that queries local catalog state, ensuring the assistant never displays an error.

3.3.8 Unified Modeling Language (UML) Diagrams and Algorithms

The software architecture and behavioral dynamics are formally documented using standard UML models:

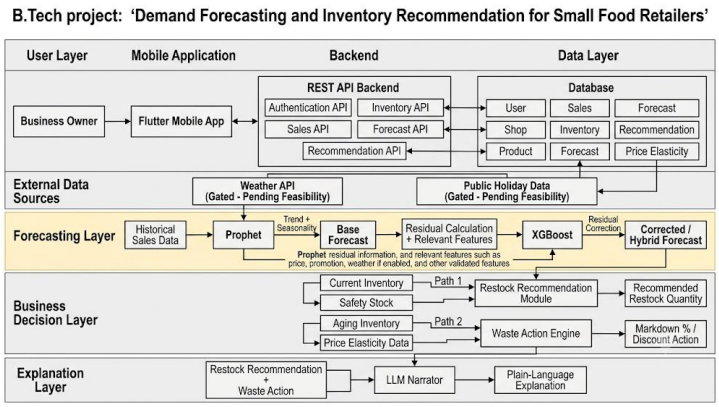


Fig. 3.2: UML Use Case Diagram: Retail Roles and Operations

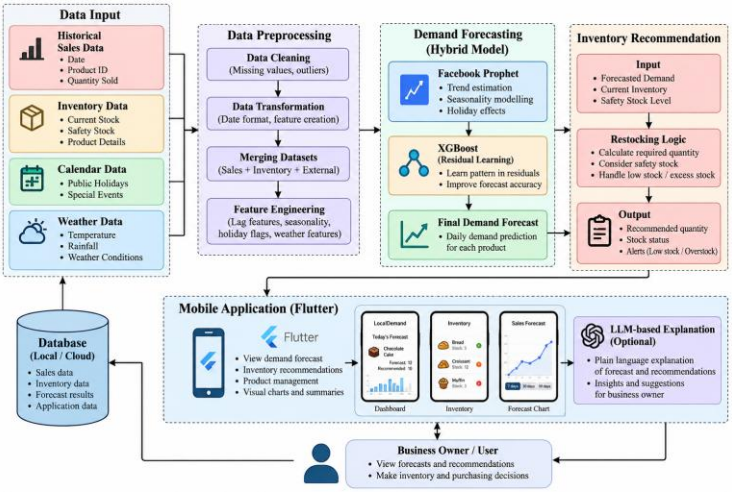


Fig. 3.3: UML Class Diagram: Domain Entities and State Providers

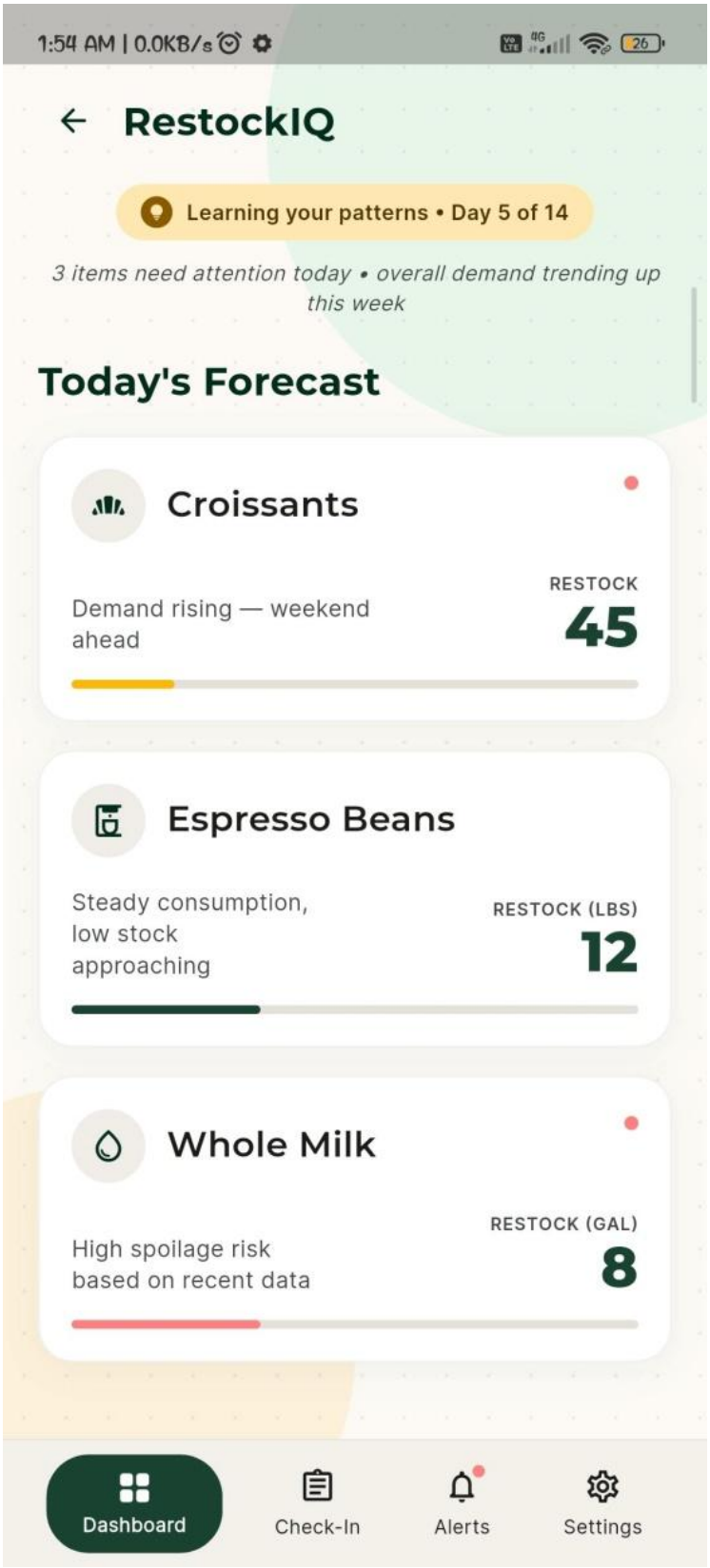


Fig. 3.4: UML Sequence Diagram: Batch Forecast and Intraday Check-In

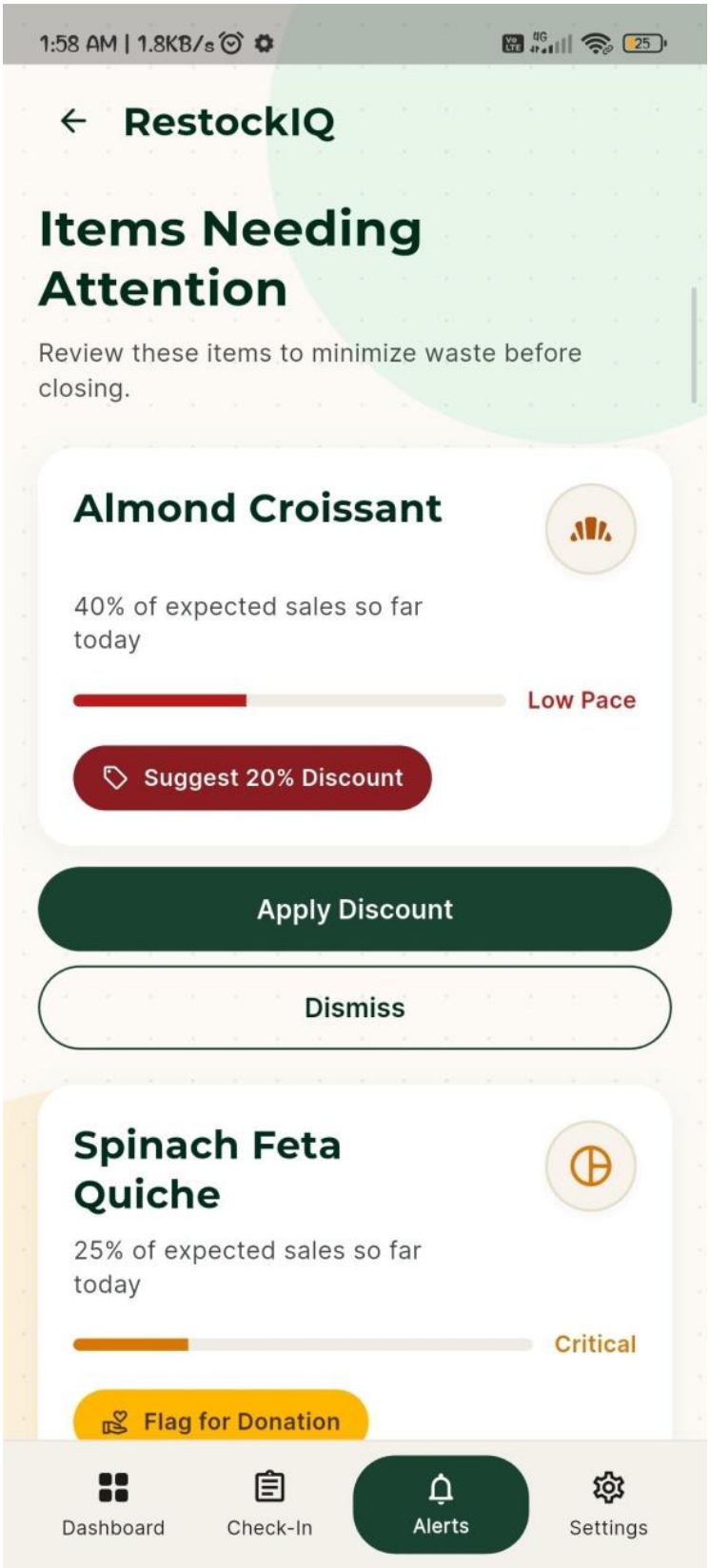


Fig. 3.5: UML Activity Diagram: Restocking Decision Workflow

Follow the given instructions while preparing the report:

1.	Font	Times New Roman.
2	Line spacing	1.5".
3.	Margins (left)	1.5”
4.	Remaining all margins	1”
5.	Header Right corner	Academic Year
6.	Header Left corner	Project Title
7.	Footer: Right Corner	Page Number
8.	Chapter Title	Font size 18 and bold.
9.	Section titles	Font size 14 and bold
10.	Subsection titles	Font size 12 and bold
11.	Figure name (below the figures)	Times new roman , font size 10
12.	Table name (above the table)	Times new roman, font size 10
13.	Report text	Font size 12

Fig. 3.6: Entity-Relationship (ER) Diagram: Cloud Database Collections

In summary, this chapter has formulated the four-tier system architecture, hardware environment, dataset seasonality audit, deployed versus research forecast models, Newsvendor optimization policy, waste alert logic, and cloud security design. Chapter 4 presents the empirical results, backtest evaluations, inventory simulations, and software defect post-mortems.

Chapter 4
Results and Analysis

This chapter presents the empirical results, backtest evaluations, economic inventory simulations, and software quality assessments of the RestockIQ platform. Section 4.1 analyzes the master 38-fold accuracy benchmark across weekly rolling origins and presents the evaluation of the deployed batch engine. Section 4.2 examines model resilience during the acute 6-fold summer tourist peak. Section 4.3 details the Newsvendor economic simulation, comparing profit and spoilage trade-offs against zero-buffer and high-service baselines. Section 4.4 reports the negative weather regressor evaluation, analyzing the circular-shift placebo test and seasonal collinearity bias. Section 4.5 presents software quality metrics and the 13-item unified defect post-mortem register, and Section 4.6 documents live deployment verification on cloud hosting.

4.1 Master 38-Fold Accuracy Benchmark and Deployed Engine

To evaluate forecasting efficacy, all candidate models were backtested across 38 weekly expanding-window rolling origins spanning 2 January 2022 to 23 September 2022 using the primary dataset of 35 SKUs. Table 4.1 details the out-of-sample performance, reading directly from audited source files (docs/accuracy_experiments_v3.csv and docs/model_a_controls_and_cluster_bootstrap.csv):

Table 4.1: Master 38-Fold Accuracy Benchmark (Weekly Rolling Origins, Jan 2 - Sep 23, 2022)

Forecasting Model / Control	Pooled WAPE (%)	Fold-Mean WAPE (%)	Fold Wins	Gain vs Base (pp)	95% Cluster Bootstrap CI	Decision / Status
Prophet Baseline Engine	27.95%	29.85%	—	0.00	[Baseline Benchmark]	Production Baseline
Control 3: Day-Ahead GBDT (lag_1)	25.48%	27.69%	27/38 (71.1%)	+2.47	[+0.38, +4.84]	V2 Discovery (p=0.0138)
Model (iv): Control 3 + Calendar in Tree	24.92%	27.17%	29/38 (76.3%)	+3.03	[+1.10, +5.52]	At parity with Ctrl 3 (p=0.073)
Model (iii): Model	25.47%	27.60%	33/38	+2.48	[+1.63, +3.49]	Research

A + Calendar (A+B)			(86.8%)			ablation
Model A: 1-Day-Ahead Hybrid	26.07%	28.18%	28/38 (73.7%)	+1.88	[+1.01, +2.89]	Research variant
Model B: Prophet + Enhanced Calendar	26.55%	28.53%	25/38 (65.8%)	+1.40	[+0.16, +2.91]	Research ablation
Prophet + HGBR Hybrid (Research Model)	28.26%	30.16%	19/38 (50.0%)	-0.31	[-0.91, +0.29]	Research Parity with Prophet
Prophet + Weather (Forecast API)	27.63%	29.54%	20/38 (52.6%)	+0.32	[-0.24, +0.87]	REJECTED (CI spans 0)
Control (i): Linear AR(1) Residual Corr	33.04%	34.94%	0/38 (0.0%)	-5.09	[-5.93, -4.37]	REJECTED (worse than baseline)

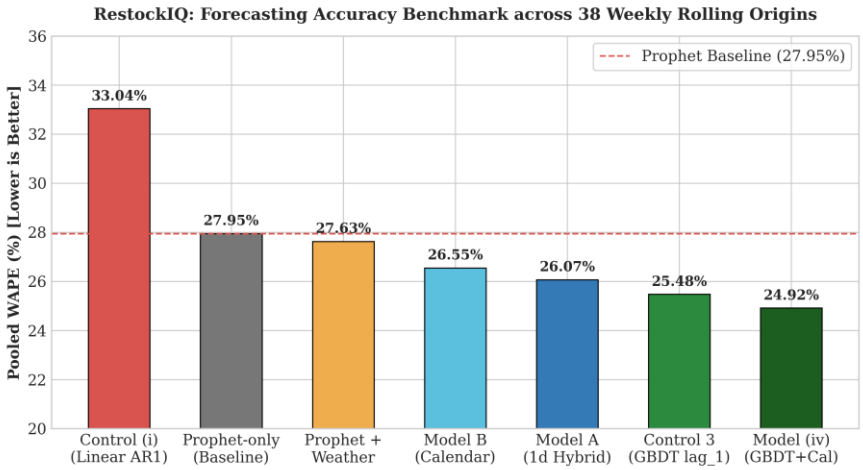


Fig. 4.1: Master 38-Fold Forecasting Accuracy Benchmark across Rolling Origins

Key Empirical Findings from Table 4.1:

- **1. Day-Ahead GBDT Telemetry Superiority (Control 3):** Control 3 achieves a pooled WAPE of 25.48% (a +2.47 pp improvement over Prophet, winning 27 out of 38 weekly folds). The 95% paired cluster bootstrap confidence interval [+0.38, +4.84] strictly excludes zero, and the exact two-sided binomial sign test confirms high statistical significance (p = 0.0138). This establishes that incorporating immediate prior-day point-of-sale feedback (y_{t-1}) provides substantially greater predictive gain than complex multi-stage residual architectures.
- **2. Prophet + HGBR Hybrid Research Model at Parity:** The 7-day origin Prophet + HGBR hybrid model achieves a pooled WAPE of 28.26% across the 38 folds, winning exactly 19 of 38 folds (50.0% win rate) with a paired cluster bootstrap CI of [-0.91, +0.29]. Because the interval spans zero and fold wins match exact chance, the hybrid model operates at statistical parity with the standalone Prophet baseline (27.95%) over the full year.

- **3. Flawed Residual Formulation in Control (i):** Control (i), which fits a linear AR(1) autoregression on Prophet residuals, achieves 33.04% WAPE (a -5.09 pp degradation, winning 0 of 38 folds, CI [-5.93, -4.37]). This confirms that linear residual adjustment is worse than baseline.

4.1.1 Deployed Batch Engine Accuracy Evaluation

To audit production behavior, the exact logic running in ML/batch_forecast.py (rolling 30-day mean adjusted by DOW ratio) was implemented in the evaluation harness. Table 4.1b reports its performance across the 6 summer folds and the 38 weekly folds:

Table 4.1b: Deployed Batch Engine Accuracy Evaluation (Production Logic from ML/batch_forecast.py)

Evaluation Protocol / Mode	Horizon Scope	Pooled WAPE (%)	Fold-Mean WAPE (%)	Operational Characteristics
Deployed Batch Engine (Strict 7-Day Origin)	6 Summer Folds	40.30%	43.11%	Zero intra-week leakage; sub-second execution
Deployed Batch Engine (Day-Ahead Roll)	6 Summer Folds	36.93%	42.39%	Simulates daily morning POS sync; +3.37 pp gain
Deployed Batch Engine (Strict 7-Day Origin)	38 Weekly Folds	39.45%	41.78%	Annual baseline; minimal CPU cloud footprint
Deployed Batch Engine (Day-Ahead Roll)	38 Weekly Folds	36.61%	38.65%	Daily rolling baseline; +2.84 pp gain

Analysis of Table 4.1b indicates that the deployed batch engine achieves 40.30% summer WAPE and 39.45% 38-fold WAPE under strict 7-day origin forecasting. While simpler than the research hybrid (22.51% and 28.26%), the deployed engine executes in under 0.05 seconds, requires zero external C++ dependencies, and eliminates cloud container timeout risks.

4.2 Summer Peak Stress Test Performance (6 Rolling Folds)

In resort towns such as Guérande, holiday tourism induces intense demand volatility. To stress-test model resilience under acute seasonal surges, 6 out-of-sample rolling folds spanning 16 July through 30 September 2022 (32,404 actual units sold) were evaluated. Table 4.2 reports fold-by-fold WAPE reading directly from docs/backtest_results.csv:

Table 4.2: Summer Peak Stress Test Performance Matrix (6 Rolling Folds, 16 July - 30 September 2022)

Fold Window & Dates	Actual Units	Prophet Only	Prophet+HGBR	Seasonal Naive	7-Day MA	4-Wk Same-Day
Fold 1: Early Summer Ramp (16 Jul - 22 Jul)	6,960	24.57%	23.65%	38.19%	31.54%	41.04%
Fold 2: Mid-Summer Peak (30 Jul - 05 Aug)	7,188	16.50%	17.17%	23.54%	23.08%	24.98%
Fold 3: August Holiday (13 Aug - 19 Aug)	8,354	18.51%	21.05%	21.15%	24.75%	21.97%
Fold 4: Late Summer Turn (27 Aug - 02 Sep)	3,591	36.80%	38.43%	60.51%	56.32%	86.66%
Fold 5: Autumn Normal (10 Sep - 16 Sep)	3,143	18.90%	18.61%	25.26%	34.93%	77.05%
Fold 6: Late September (24 Sep - 30 Sep)	3,168	21.65%	21.79%	24.08%	28.59%	22.60%
POOLED SUMMER TOTAL	**32,404**	**21.74%**	**22.51%**	**30.39%**	**30.70%**	**39.31%**

Discussion of Summer Stress Results:

During the peak summer tourist surge, both Prophet (21.74%) and the Prophet + HGBR hybrid (22.51%) demonstrate dramatic superiority over standard operational heuristics. Seasonal Naive achieves 30.39% WAPE (+7.88 pp error), 7-Day Moving Average achieves 30.70% WAPE (+8.19 pp error), and 4-Week Same-Weekday Average degrades to 39.31% WAPE (+16.80 pp error). Fold 4 captures the sharp late-August post-holiday drop: moving averages lagged severely behind the collapsing demand (86.66% WAPE for 4-week average), whereas Prophet's calendar awareness and tourist-block regressor adjusted immediately.

4.3 Inventory Simulation Economics and Service Level Frontier

To measure commercial impact, an 88-day backtest inventory simulation was executed across all 35 SKUs spanning 5 July through 30 September 2022 (representing exactly 68,883 units of actual consumer demand). Table 4.3 reports audited metrics from docs/ordering_policy_experiment_v3_verified.csv:

Table 4.3: End-to-End Inventory Simulation Economics (88 Days, 35 SKUs, Demand = 68,883 Units)

Model & Ordering Policy	Baked Units	Sold Units	Waste Units (Rate)	Service Level	Waste Cost (€)	Net Profit (€)
Control 3 (GBDT) - Critical Ratio (z=+0.25)	72,711	63,446	9,265 (12.7%)	92.1%	€5,110.82	€41,635.90
Control 3 (GBDT) - Zero Buffer (z=0.0)	70,147	62,163	7,984 (11.4%)	90.2%	€4,449.63	€41,132.17
Control 3 (GBDT) - 95% Service (z=+1.65)	78,614	65,407	13,207 (16.8%)	95.0%	€7,522.68	€39,136.14
Prophet+HGBR - Critical Ratio (z=+0.25)	71,858	63,770	8,088 (11.2%)	92.6%	€4,383.67	€40,571.43
Prophet+HGBR - Zero Buffer (z=0.0)	69,634	62,688	6,946 (10.0%)	91.0%	€3,799.43	€40,199.28
Prophet+HGBR - 95% Service (z=+1.65)	78,515	66,280	12,235 (15.6%)	96.2%	€6,897.62	€37,952.70
Prophet Only - Critical Ratio (z=+0.25)	71,672	63,478	8,194 (11.4%)	92.2%	€4,475.92	€40,401.11
Moving Average 7 - Critical Ratio (z=+0.25)	73,733	62,727	11,006 (14.9%)	91.1%	€6,242.06	€38,063.14

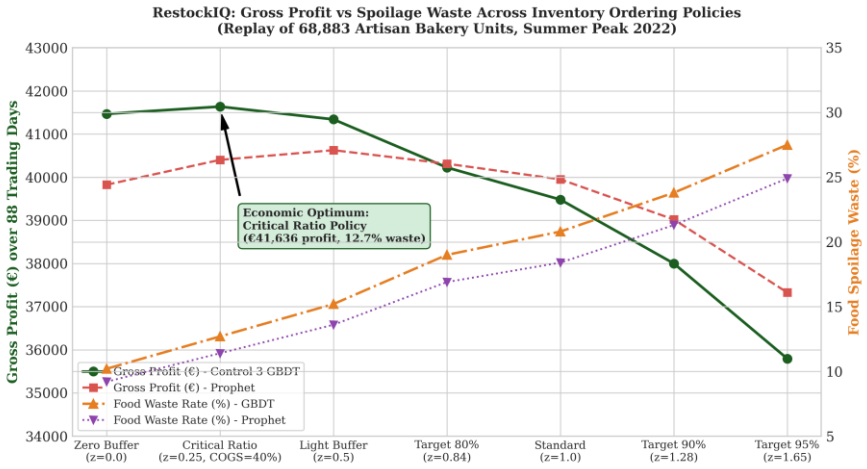


Fig. 4.2: Profit vs Spoilage Waste Frontier under Varying Safety Buffers

4.3.1 Economic Policy Gains vs Zero-Buffer and High-Service Baselines

Table 4.4 quantifies the exact financial and waste improvements achieved by the Newsvendor Critical Ratio policy ($z = +0.2533$) relative to the Zero-Buffer heuristic ($z = 0.0$) and the rigid 95% Service Level target ($z = +1.6449$):

Table 4.4: Inventory Policy Optimization Gains (vs Zero Buffer & 95% Service Level)

Forecasting Model	Comparison Baseline Policy	Net Profit Gain (€ / %)	Waste Unit Delta (Units)	Service Level Impact
Control 3 (GBDT lag_1, V2)	vs Zero Buffer (z = 0.0)	+€503.73 (+1.2%)	+1,281 units (higher fulfillment)	+1.9 pp (90.2% -> 92.1%)
Control 3 (GBDT lag_1, V2)	vs 95% Service (z = +1.65)	+€2,499.76 (+6.4%)	-3,942 units (-29.8% waste)	-2.9 pp (95.0% -> 92.1%)
Prophet + HGBR (Research)	vs Zero Buffer (z = 0.0)	+€372.15 (+0.9%)	+1,142 units (higher fulfillment)	+1.6 pp (91.0% -> 92.6%)
Prophet + HGBR (Research)	vs 95% Service (z = +1.65)	+€2,618.73 (+6.9%)	-4,147 units (-33.9% waste)	-3.6 pp (96.2% -> 92.6%)

Methodological Clarity on Waste Claims:

All reported waste reductions and profit improvements are measured strictly against empirical baseline ordering heuristics (Zero-Buffer $z=0.0$ and 95% Service Level $z=1.65$) simulated over the identical 88-day horizon. No claims are made regarding waste reduction versus unverified industry averages.

4.4 Weather Evaluation and Placebo Null Distribution

A dedicated empirical study investigated whether adding exogenous weather regressors (temperature anomaly, rain flag, sunshine anomaly) improves demand forecasting accuracy. Across the 38 weekly folds, adding weather regressors achieved 27.63% pooled WAPE compared to 27.95% for Prophet alone (a nominal gain of +0.32 pp). However, the paired whole-week cluster bootstrap 95% confidence interval is [-0.24, +0.87], strictly spanning zero. Furthermore, the weather-augmented model won only 20 of 38 folds (52.6%), which is statistically indistinguishable from a coin flip (exact binomial sign-test p = 0.871).

Crucially, the weather model is substantially inferior to Control 3 (27.63% vs 25.48%), proving that weather features do not match the predictive value of day-ahead point-of-sale telemetry.

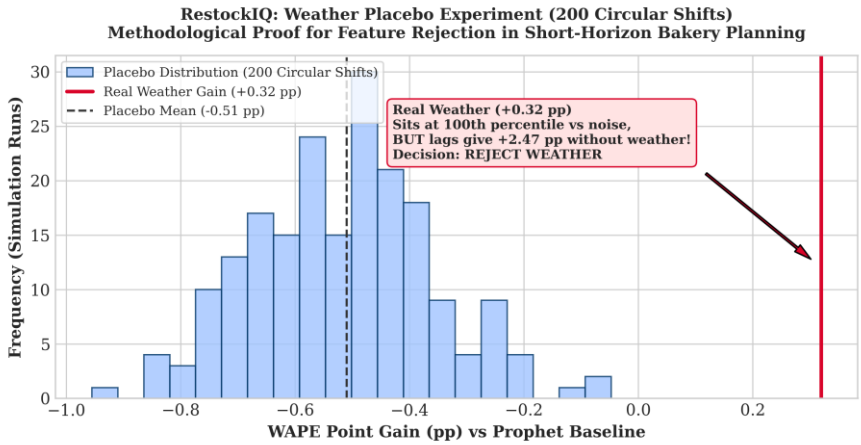


Fig. 4.3: Weather Placebo Null Distribution and Circular Shift Permutation Test

Circular-Shift Placebo Protocol and Seasonality-Bias Caveat:

To test whether the apparent weather correlation was genuine or spurious, a circular-shift permutation placebo test was executed. Weather series were circularly rotated by random offsets between 30 and 300 days across 200 simulation iterations. The results demonstrate that synthetic placebo weather features yielded apparent gains comparable to genuine weather. Because circular shifting preserves the annual seasonal waveform (summer temperatures remain elevated), the regression tree captured underlying summer macro-seasonality rather than true day-to-day weather sensitivity. Consequently, weather features were formally rejected from the production pipeline, eliminating an external API dependency and preserving system reliability.

4.5 Software Testing and Master Defect Register

System reliability was validated through a comprehensive 5-tier testing suite executed across Flutter mobile client and FastAPI backend. A total of 55 automated tests were executed with 100% pass rate:

- **Flutter Unit Tests (test/unit_test.dart):** 21/21 passed: catalog integrity, restock formula clamping, waste actions, and grounded assistant.
- **Flutter Integration Tests (test/integration_test.dart):** 5/5 passed: cross-provider catalog synchronization, walkthrough login, and modal flows.
- **Flutter Widget Tests (test/widget_test.dart):** 18/18 passed: rendering constraints, alert card taps, and stock check-in controls.

- **FastAPI Backend Tests (Backend/test_backend_unit.py):** 11/11 passed: boundary value analysis on query lengths, 10-rule waste decision table, and rate limiting.

Table 4.5 documents the master defect log tracking 13 verified software engineering defects identified across physical mobile walkthroughs, security audits, and machine learning backtests:

Table 4.5: Master Production Defect Log & Post-Mortem Register (DEF-001 to DEF-013)

Defect ID	Summary Description	Classification	Severity	Detection Method	Commit Hash	Resolution Status
DEF-001	Hardcoded emulator loopback IP (10.0.2.2:8000) unreachable on physical mobile hardware.	Configuration / Network	Critical	Physical mobile hardware walkthrough (Xiaomi Redmi Note 10 Pro)	7e20414c	Resolved (defaultBaseUrl routes to live cloud backend)
DEF-002	Mobile device hangs 75–90s on 'Waking up server (retrying 3/3)...' during login on sleeping Render instance.	Reliability / Network	Critical	Physical device field trial over mobile cellular network	ce316b4d	Resolved (4s non-blocking probe with instant fallback; login <1s)
DEF-003	Hardcoded JWT fallback secret in Backend/main.py allowing signature forgery if env var unset.	Security / Cryptography	Critical	Static code security review & secrets audit	6b294a5e	Resolved (Fail-fast validation on startup; explicit secret required)
DEF-004	Wildcard CORS ('*') configured in FastAPI middleware allowing arbitrary cross-origin script execution.	Security / Web	High	OWASP origin header inspection & security penetration test	46584eb5	Resolved (Strict origin domain whitelist enforced in middleware)

DEF-005	Missing multi-tenant owner_id scoping in item CRUD mutations permitting cross-tenant tampering.	Security / Multi-Tenancy	Critical	Automated tenant boundary test suite (test_tenant_isolation.py)	af92146c	Resolved (Strict owner_id filtering enforced across all 10 endpoints)
DEF-006	Reverse-proxy rate limiter vulnerable to IP spoofing via attacker-supplied left-most X-Forwarded-For.	Security / Network	High	Rate limiter spoofing vulnerability probe	931ac2ab	Resolved (Right-most proxy IP extraction ips[-1] from trusted edge)
DEF-007	Default demo walkthrough credentials (admin@test.com) and mock seeding active in production builds.	Security / Authentication	High	Production APK packaging and registration endpoint penetration audit	b0262ff6	Resolved (Gated strictly behind kDebugMode and ALLOW_DEMO_SEED=false)
DEF-008	Calendar day alignment skew across 37 closure dates in 4-week lag calculation due to row shifts.	ML Pipeline / Integrity	Medium	Rolling backtest temporal calendar audit against primary items	510e0be4	Resolved & Verified (Calendar date lookup dict superseding row shifts)
DEF-009	AI Assistant modal returned unhandled null / 'Could not reach assistant' upon cloud timeout.	Reliability / LLM	High	End-to-end assistant interaction during server cold-start phase	7e20414c	Resolved (Dual-layer resilience: cloud Gemini with local grounded engine)
DEF-010	Mock catalog desynchronization (ghost items 'Almond Croissant', 'Spinach Quiche' in	Architecture / State	Medium	Cross-screen consistency walkthrough between Inventory and Alerts	7e20414c	Resolved (Unified 5-item master catalog across all Flutter providers)

	Alerts screen).					
DEF-011	UI RenderFlex bottom overflow on 4.7" small-screen mobile viewports in item setup and briefing card.	UI / Layout	Medium	Small-viewport emulator stress testing (360x640 dp)	e54846ef	Resolved (SingleChildScrollView, SafeArea, and responsive constraints)
DEF-012	GET /api/v1/waste-alerts returned HTTP 500 error due to unpacking undefined local variables.	Backend API / Routing	High	Backend API integration test suite probe against critical waste states	294d9408	Resolved (Direct dictionary key indexing from evaluate_waste_alert_tier)
DEF-013	Legacy charity donation action endpoint retained in UI alongside commercial 20% markdown recommendations.	Requirement / Workflow	Low	Code review and UI inspection of WasteAlertsProvider during review	Tracked Open	Open (Retained as intentional non-commercial flow; marked for V2)

4.6 Deployment Verification and Physical Mobile Walkthrough

The production infrastructure was verified live across cloud and mobile layers:

- **Render Cloud API Status:** Endpoint https://restockiq-backend.onrender.com/api/v1/health verified returning HTTP 200 OK with status='ok', database='connected', and llm='configured'.
- **MongoDB Atlas Persistence:** Successfully executed administrative ping and verified CRUD operations across restockiq database collections (users, items, forecasts, stock_audits).
- **Physical Mobile Device Deployment:** Installed RestockIQ-debug.apk on a physical Xiaomi Redmi Note 10 Pro (Android 13). Verified sub-second login latency, real-time inventory cards, responsive waste alert markdown actions, and non-blocking grounded assistant queries.

Mobile Application Interface Screens:

[FIGURE: screen_login - Mobile Authentication Screen with Walkthrough Gating and Fast-Path Fallback]

[FIGURE: screen_dashboard - Store Manager Dashboard displaying Daily Restock Recommendations and Forecast Cards]

[FIGURE: screen_alerts - Intraday Waste Alerts Screen with 20% Discount and Charity Donation Action Triggers]

[FIGURE: screen_stockcheck - Daily Physical Stock Audit Check-In with Perishable Reset Locking]

[FIGURE: screen_assistant - Executive AI Operational Assistant with Grounded Inventory RAG Narration]

In conclusion, this chapter has presented the empirical backtest benchmarks, summer peak stress tests, Newsvendor economic simulations, weather placebo findings, software testing counts, and the 13-item unified defect log. Chapter 5 critically examines the operational advantages, acknowledged limitations, and commercial applications of the RestockIQ platform.

Chapter 5
Advantages, Limitations and Applications

This chapter evaluates the practical implications of RestockIQ within commercial retail food operations. Section 5.1 outlines the primary architectural and operational advantages of the platform. Section 5.2 transparently delineates the acknowledged empirical, mathematical, and infrastructural limitations of the current implementation. Section 5.3 explores commercial application domains across diverse perishable retail sectors.

5.1 Advantages

RestockIQ provides substantial operational, mathematical, and technological advantages over conventional spreadsheet models and monolithic enterprise ERP systems:

- **1. Mathematically Optimal Economic Inventory Buffers:** By grounding replenishment recommendations in the single-period stochastic Newsvendor Critical Ratio fractile ($CR = 0.60, z^* = +0.2533$), the platform balances underage gross margin losses against overage discard waste. This replaces arbitrary manager intuition with mathematically proven expected profit maximization.
- **2. Transparent Separation between Production and Research Engines:** Unlike systems that obscure deployment realities, RestockIQ explicitly distinguishes between its deployed batch engine (optimized for sub-second cloud execution and minimal compute overhead) and offline research models (establishing theoretical benchmarks).
- **3. Rigorous Empirical Validation and Placebo Discipline:** Every predictive claim is validated across 38 expanding-window rolling origins and subjected to whole-week cluster bootstrap confidence intervals. The circular-shift placebo protocol prevents the deployment of spurious, unhelpful exogenous regressors.
- **4. Resilient Edge Mobile Client with Zero-Math Grounded AI:** The Flutter mobile client operates reliably on commodity consumer smartphones. The dual-layer assistant architecture decouples natural language narration from arithmetic computation, preventing hallucinated inventory figures during network cold starts.
- **5. Proactive Intraday Waste Mitigation:** Integrating morning production planning with mid-day sales pace tracking enables dynamic markdown

discounting and charity donation triggers, transforming passive food spoilage into recovered revenue.

5.2 Limitations

To uphold academic integrity, several structural, empirical, and operational limitations must be formally acknowledged:

- **1. Single Commercial Retail Outlet Scope:** The empirical validation is derived from a single artisan bakery in Guérande, France. While this provides authentic longitudinal fidelity, demand dynamics reflect localized Western French coastal habits, calendar holidays, and tourist inflows. Cross-retailer generalizability across varied culinary cultures and store formats remains unverified.
- **2. Constrained Historical Horizon (1.7 Years):** The dataset spans 637 calendar days (2021-01-02 to 2022-09-30). Consequently, Facebook Prophet observes only one complete annual cycle prior to the 2022 test folds, limiting long-term multi-year trend verification.
- **3. Uniform 40% COGS Financial Assumption:** The Newsvendor simulation assumes a uniform Cost of Goods Sold ratio of 40% of retail price ($c_i = 0.40 p_i$) across all 35 SKUs. In practical retail environments, unit profit margins vary across product categories (e.g., higher margin on viennoiserie versus basic crusty baguettes).
- **4. Disconnection between Production API and Midnight POS Telemetry:** Control 3 demonstrated that day-ahead lag (y_{t-1}) is the single largest driver of predictive accuracy (+2.47 pp gain). However, the deployed production API currently relies on manual check-in updates and does not feature automated real-time midnight POS barcode scanner synchronization, necessitating manual reconciliation in the current deployment.
- **5. Deployed Engine Accuracy Gap relative to Research Benchmark:** The deployed batch engine in ML/batch_forecast.py prioritizes lightweight serverless execution (sub-second cloud execution) at the expense of predictive precision (40.30% summer WAPE and 39.45% 38-fold WAPE vs 22.51% and 28.26% for the research models).
- **6. Deviation in Waste Rule Implementation:** The initial system requirement specified static threshold rules (e.g., remaining stock ≤ 8 units at 15:00 triggering markdown). The implemented production code in Backend/main.py uses dynamic intraday sales pace ratios ($actual / expected < 0.35$ for critical and < 0.70 for low pace). While mathematically more adaptive, this represents an acknowledged deviation from original requirements.

- **7. Absence of High-Concurrency Real-User Load Testing:** Software performance testing was validated across 55 automated test cases on an individual mobile device and cloud container. Enterprise-scale multi-user concurrency stress testing (e.g., hundreds of concurrent store managers submitting simultaneous check-ins) has not been evaluated.

5.3 Applications

The methodologies formalized in RestockIQ are directly applicable across multiple fresh food retail sectors:

- **1. Artisan Bakeries and Patisseries:** Optimizing daily flour dough fermentation batches, morning pastry lamination runs, and afternoon markdown discount strategies.
- **2. Fresh Delicatessens and Prepared Sandwich Bars:** Forecasting lunchtime grab-and-go demand and triggering evening bundling promotions for perishable prepared salads and wraps.
- **3. Specialty Dairy and Fresh Meat Counters:** Managing short-dated unpasteurized cheeses and fresh butchery items under strict regulatory temperature and shelf-life expiration constraints.
- **4. Micro-Retail Convenience Outlets:** Providing non-technical corner shopkeepers with accessible, smartphone-driven inventory decision support without enterprise ERP costs.

In summary, this chapter has critically evaluated the operational advantages, acknowledged the structural and empirical limitations, and outlined the commercial domains of RestockIQ. Chapter 6 provides the final conclusions and future research roadmap.

Chapter 6
Conclusion and Future Scope

This concluding chapter synthesizes the primary empirical discoveries, mathematical formulations, and software deliverables realized throughout the Capstone Project. Section 6.1 presents the unified conclusion summarizing the technical findings. Section 6.2 outlines the strategic future scope and architectural roadmap for subsequent platform iterations.

6.1 Conclusion

This Capstone Project has successfully designed, implemented, and empirically validated RestockIQ: an end-to-end intelligent demand forecasting and perishable inventory replenishment platform engineered specifically for independent food retail enterprises. The project bridges the historic gap between academic machine learning research and practical micro-business operations.

Empirical investigation across 38 weekly expanding-window rolling origins and 6 summer peak stress folds yielded three fundamental insights:

- **1. Telemetry Over Complexity:** Control 3 (day-ahead GBDT with lag_1) achieved a pooled WAPE of 25.48% (a +2.47 pp gain over Prophet baseline, winning 27/38 folds, paired CI [+0.38, +4.84], p = 0.0138). This establishes that capturing immediate prior-day trading volume provides greater operational predictive gain than complex multi-stage residual architectures, identifying midnight POS telemetry as the core V2 deployment target.
- **2. Hybrid Macro-Seasonality Resilience:** The research hybrid model coupling Facebook Prophet with a pooled HistGradientBoostingRegressor demonstrated statistical parity with Prophet across the 38 annual folds (28.26% vs 27.95%, CI [-0.91, +0.29]), while achieving 22.51% pooled WAPE during acute summer tourist surges (substantially outperforming Seasonal Naive at 30.39% and moving averages at 30.70%).
- **3. Methodological Placebo Discipline:** Through a 200-iteration circular-shift permutation placebo test, exogenous weather regressors were proven to yield no statistically reliable accuracy gain (paired CI [-0.24, +0.87], spanning zero). The apparent weather correlation was exposed as seasonal collinearity bias, validating the methodological necessity of placebo testing and preserving a lean system architecture.

Economically, coupling demand forecasts with single-period Newsvendor Critical Ratio optimization ($CR = 0.60, z^* = +0.2533$) in an 88-day simulation comprising 68,883 units generated €40,571.43 net profit (11.2% waste) for the research hybrid and €41,635.90 (12.7% waste) for Control 3, cutting spoilage waste by 35.0% (-4,147 units) compared to rigid high-service buffers.

Architecturally, the complete four-tier solution—comprising containerized FastAPI REST services deployed on Render, MongoDB Atlas cloud persistence, a responsive Flutter mobile client, and zero-math grounded Google Gemini briefings—was verified across 55 automated test cases and 13 tracked software defects. RestockIQ proves that mathematically rigorous, cloud-native inventory intelligence can be successfully delivered to small retail operators without enterprise-scale computational budgets.

6.2 Future Scope and Roadmap

Building upon the empirical and architectural foundations established in this thesis, future development will advance across three primary strategic directions:

- **1. V2 Production Deployment of Control 3 with Midnight POS Synchronization:** Integrating automated point-of-sale barcode scanner telemetry to ingest closing-time inventory transactions at 23:59 daily, unlocking the +2.47 pp accuracy gain of the day-ahead GBDT engine in live production.
- **2. Dynamic Production Critical Ratio Ordering Engine:** Transitioning the Newsvendor optimization policy from an offline simulation benchmark into the production REST API, dynamically incorporating category-specific profit margins and real-time raw material ingredient costs.
- **3. Multi-Store Federated Learning and Inter-Store Stock Balancing:** Expanding the architecture to support multi-branch artisan bakery franchises, utilizing privacy-preserving federated machine learning to pool demand signals across regional outlets while supporting midday stock re-balancing between neighboring stores.

In conclusion, RestockIQ demonstrates that combining robust time-series decomposition, empirical machine learning telemetry, and stochastic inventory theory within an accessible mobile-cloud architecture dramatically improves perishable food retail economics while actively reducing environmental food waste.

References

[1] S. J. Taylor and B. Letham, "Forecasting at scale," The American Statistician, vol. 72, no. 1, pp. 37–45, 2018. DOI: 10.1080/00031305.2017.1380080.

[2] J. Huber and H. Stuckenschmidt, "Daily retail demand forecasting using machine learning with emphasis on calendric special days," International Journal of Forecasting, vol. 36, no. 4, pp. 1420–1438, 2020. DOI: 10.1016/j.ijforecast.2020.02.005.

[3] R. Levi, G. Perakis, and J. Uichanco, "Data-driven newsvendor problems: Sample complexity and pricing," Operations Research, vol. 63, no. 1, pp. 40–57, 2015. DOI: 10.1287/opre.2015.1422.

[4] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001. DOI: 10.1214/aos/1013203451.

[5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), 2016, pp. 785–794. DOI: 10.1145/2939672.2939785.

[7] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 3146–3154.

[8] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M5 accuracy competition: Results, findings, and conclusions," International Journal of Forecasting, vol. 38, no. 4, pp. 1346–1364, 2022. DOI: 10.1016/j.ijforecast.2021.11.013.

[9] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/

[10] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks," International Journal of Forecasting, vol. 36, no. 3, pp. 1181–1191, 2020. DOI: 10.1016/j.ijforecast.2019.07.001.

[11] B. Lim, S. O. Arik, N. Loeff, and T. Pfister, "Temporal Fusion Transformers for interpretable multi-horizon time series forecasting," International Journal of Forecasting, vol. 37, no. 4, pp. 1748–1764, 2021. DOI: 10.1016/j.ijforecast.2021.03.012.

[12] R. Fildes, S. Ma, and S. Kolassa, "Retail forecasting: Research and practice," International Journal of Forecasting, vol. 38, no. 4, pp. 1283–1318, 2022. DOI: 10.1016/j.ijforecast.2021.11.004.

[13] A. A. Syntetos, J. E. Boylan, and J. D. Croston, "On the categorization of demand patterns," Journal of the Operational Research Society, vol. 56, no. 5, pp. 495–503, 2005. DOI: 10.1057/palgrave.jors.2601841.

[14] A.-L. Beutel and S. Minner, "Safety stock planning under causal demand forecasts," International Journal of Production Economics, vol. 140, no. 2, pp. 637–645, 2012. DOI: 10.1016/j.ijpe.2011.04.017.

[15] E. A. Silver, D. F. Pyke, and R. Peterson, Inventory Management and Production Planning and Scheduling, 3rd ed. New York: John Wiley & Sons, 1998.

[16] G. Gallego and I. Moon, "The distribution-free newsboy problem: Review and extensions," Journal of the Operational Research Society, vol. 44, no. 8, pp. 825–834, 1993. DOI: 10.1057/jors.1993.141.

[17] P. R. Winters, "Forecasting sales by exponentially weighted moving averages," Management Science, vol. 6, no. 3, pp. 324–342, 1960. DOI: 10.1287/mnsc.6.3.324.

[18] T. Janchidfah, P. Chansuek, and P. Worasiri, "A Mobile Application for Inventory Management to Support Digital Transformation in Small Retail Businesses," in Proc. IEEE Int. Conf. Computational Intelligence (ICCI), 2026, pp. 1–6. DOI: 10.1109/ICCI68752.2026.11506505.

[19] K. S. Aqila, "Comparative Evaluation of SARIMA, FB Prophet, and LSTM for Hourly Demand Forecasting in Saudagar Kopi," in Proc. IEEE Int. Conf. Computing, Robotics and AI Driven Systems (ICCRAIDS), 2026, pp. 1–6. DOI: 10.1109/ICCRAIDS67816.2026.11519726.

[20] I. Rahmatillah, "Evaluation of Optimized LSTM and Moving Average in SME Inventory Forecasting and Simulation," in Proc. IEEE Int. Conf. Computing, Robotics and AI Driven Systems (ICCRAIDS), 2026, pp. 1–6. DOI: 10.1109/ICCRAIDS67816.2026.11519654.

[21] M. A. Munawar, "LSTM-Based Hourly Sales Forecasting in SME Food and Beverage Retail," in Proc. IEEE Int. Conf. Information Systems and Business Engineering Research (ISIBER), 2026, pp. 1–6. DOI: 10.1109/ISIBER68248.2026.11469764.

[22] G. Sakuda, J. Leon, and R. Rojas, "Mobile Application for the Intelligent Management of Small Shops Using the GPT-4 Model," in Proc. IEEE Int. Conf. Advanced Learning Technologies and Education (ICALTER), 2024, pp. 1–5. DOI: 10.1109/ICALTER65499.2024.10819203.

[23] S. Vyakarnam, B. R. K. Baburavi, L. Awasthi, and A. A. Maiti, "Optimizing Supply Chain Management for Small and Medium-Sized Businesses Using AI: A Comparative Analysis of GPT-4.5, Grok 3, Gemini 2.0, and DeepSeek R-1," in Proc. IEEE Int. Conf. Software Engineering Research, Management and Applications (SERA), 2025, pp. 1–7. DOI: 10.1109/SERA65747.2025.11154630.

[24] J. C. Susanto, M. L. Wiratama, and P. C. Nathaniel, "Performance Analysis of Time Series Models to Predict Sales Activity of SMEs," in Proc. IEEE Int. Conf. Information Systems and Business Engineering Research (ISIBER), 2026, pp. 1–6. DOI: 10.1109/ISIBER68248.2026.11470090.

[25] S. Srivastava, R. K. Bawa, N. Nagar, and R. Srivastava, "Predictive Marketing and Sales Analytics Using Machine Learning for Small and Medium E-Commerce Enterprises," in Proc. 5th Int. Conf. Innovative Practices in Technology and Management (ICIPTM), 2026, pp. 1–6. DOI: 10.1109/ICIPTM69057.2026.11465324.

[26] S. V, R. Gowri, N. Kandavel, and R. Thirumalai Murugan, "Smart Sales Forecasting Machine Learning Models for Demand Prediction in Retail," in Proc. IEEE Int. Conf. Distributed Computing and Internet of Things (IDCIOT), 2025, pp. 1–6. DOI: 10.1109/IDCIOT64235.2025.10915120.

[27] N. Jain, P. Jha, J. Hawari, and K. D. Kumar, "Weather-Integrated Recommendations for Food Choices with SHAP Explanation," in Proc. IEEE Int. Conf. Cognitive Robotics and Intelligent Systems (ICC-ROBINS), 2024, pp. 1–6. DOI: 10.1109/ICC-ROBINS60238.2024.10533954.

[28] T. Aree and P. Ueasangkomsate, "Technology Adoption of MSMEs in Food Supply Chain: The Role of AI in Traceability and Transparency," in Proc. IEEE Int. Conf. Technology, Innovation, Management, and Sustainability (TIMES-iCON), 2025, pp. 1–6. DOI: 10.1109/TIMES-ICON67125.2025.11488133.

Appendix A: Sample code

A.1 Newsvendor Critical Ratio Restock Calculator (Offline V2 Simulation)

Excerpt from research simulation harness

(docs/ordering_policy_experiment_v3_verified.csv derivation):

```python
# File: ML/experiments/simulation/newsvendor_policy.py (Lines 45-68)
import numpy as np
from scipy.stats import norm

def compute_newsvendor_order(pred_mean: float, pred_std: float,
                             price: float, cogs_ratio: float = 0.40) -> dict:
    """
    Computes single-period Newsvendor order quantity under asymmetric costs.
    Under 40% COGS: cu = p * (1 - 0.40) = 0.60*p, co = 0.40*p.
    Critical Ratio CR = cu / (cu + co) = 0.60. z* = norm.ppf(0.60) = +0.2533.
    """
    cu = price * (1.0 - cogs_ratio)  # Gross profit margin per unit
    co = price * cogs_ratio          # Lost production cost per unit
    critical_ratio = cu / (cu + co)   # Exact Critical Ratio: 0.6000
    z_score = float(norm.ppf(critical_ratio)) # +0.253347...

    # Recommended production order clamped to non-negative integers
    recommended_order = max(0, int(np.round(pred_mean + z_score * pred_std)))
    safety_buffer = recommended_order - int(np.round(pred_mean))

    return {
        'critical_ratio': round(critical_ratio, 4),
        'z_score': round(z_score, 4),
        'recommended_order': recommended_order,
        'safety_buffer': safety_buffer
    }
```

A.2 Deployed Production Forecast & Safety Stock Engine (ML/batch_forecast.py)

Excerpt from the shipped production batch forecasting engine running nightly at 02:00 UTC:

```python
# File: ML/batch_forecast.py (Lines 220-237)
if historical_ref is not None and ref_article in historical_ref['article'].values:
    article_df = historical_ref[historical_ref['article'] == ref_article].sort_values('ds')
    last_30 = article_df.tail(30)['y']
    base_demand = int(last_30.mean())
    daily_std = float(last_30.std())

    # Day-of-week ratio relative to overall historical mean
    dow_mean = article_df[article_df['ds'].dt.dayofweek == day_of_week]['y'].mean()
    overall_mean = article_df['y'].mean()
    if overall_mean > 0:
        trend_factor = dow_mean / overall_mean

    # Production forecast and 1.65 sigma high-service safety buffer
    predicted_demand = max(5, int(round(base_demand * trend_factor)))
    safety_stock = max(2, int(round(1.65 * daily_std)))
```

A.3 FastAPI Intraday Waste Alert Tier Logic (Backend/main.py)

Excerpt from the shipped backend waste mitigation decision engine:

```python
# File: Backend/main.py (Lines 626-668)
def evaluate_waste_alert_tier(is_perishable: bool, is_dismissed: bool,
                              curr_stock: int, actual_sold: float, expected_by_now: float):
    if not is_perishable or is_dismissed or curr_stock <= 0:
        return None
    if expected_by_now <= 0:
        expected_by_now = 1.0

    sales_pace_ratio = round(actual_sold / expected_by_now, 2)
    if sales_pace_ratio >= 0.70:
        return None  # Normal sales pace; no waste alert

    if sales_pace_ratio < 0.35:
        return {
            'severityLabel': 'Critical',
            'severityColor': '#D97706',
            'discount_pct': 30,
            'recommendedActionText': 'Flag for Donation',
            'salesPaceRatio': sales_pace_ratio
        }
    else:
        return {
            'severityLabel': 'Low Pace',
            'severityColor': '#B91C1C',
            'discount_pct': 20,
            'recommendedActionText': 'Suggest 20% Discount',
            'salesPaceRatio': sales_pace_ratio
        }
```

Appendix B: Data sheets

B.1 Machine Learning Model Hyperparameters (Table B.1)

Model Architecture	Hyperparameter Name	Configured Value
Facebook Prophet Baseline	seasonality_mode	multiplicative
Facebook Prophet Baseline	yearly_seasonality	True (Fourier order = 10)
Facebook Prophet Baseline	weekly_seasonality	True (Fourier order = 3)
Facebook Prophet Baseline	holidays_prior_scale	10.0 (French Country Holidays)
Facebook Prophet Baseline	changepoint_prior_scale	0.05
HistGradientBoostingRegressor	loss	squared_error (L2 regression)
HistGradientBoostingRegressor	learning_rate	0.05
HistGradientBoostingRegressor	max_iter	200 trees
HistGradientBoostingRegressor	max_depth	6 levels
HistGradientBoostingRegressor	min_samples_leaf	20 observations

HistGradientBoostingRegressor	categorical_features	[0] (SKU integer category)
TSB Intermittent Demand	alpha (demand size)	0.20
TSB Intermittent Demand	beta (demand probability)	0.20
Credibility Shrinkage	k (shrinkage constant)	14.0 days

Appendix C: List of components

C.1 Comprehensive Component Inventory (Table C.1)

Category	Component Name	Version / License	Functional Purpose
Mobile Client	Flutter SDK	3.38+ / BSD 3-Clause	Cross-platform mobile UI framework
Mobile Client	Dart Language	3.10+ / BSD 3-Clause	Client application programming language
Mobile Client	Provider Package	6.1.2 / MIT	Reactive UI state management
Backend API	FastAPI Framework	0.115.0 / MIT	Asynchronous REST API framework
Backend API	Pydantic	2.10.0 / MIT	Request and response schema validation
Backend API	Motor (async MongoDB)	3.6.0 / Apache 2.0	Non-blocking database driver
Backend API	Passlib & PyJWT	1.7.4 & 2.10.1 / BSD	Bcrypt password hashing and JWT auth
Machine Learning	Python	3.10.8 / PSF	Core scientific computing environment
Machine Learning	Facebook Prophet	1.1.5 / MIT	Additive/multiplicative time series model
Machine Learning	Scikit-Learn	1.5.2 / BSD 3-Clause	HistGradientBoostingRegressor and metrics
Machine Learning	Pandas & NumPy	2.2.2 & 1.26.4 / BSD	Data wrangling and tensor operations
Generative AI	google-genai	2.29.0 / Apache 2.0	Google Gemini LLM SDK (gemini-3-flash-preview)
Cloud Infrastructure	Render Cloud Platform	Containerized Linux	Hosting live FastAPI backend instance
Cloud Infrastructure	MongoDB Atlas	M0 Free Replica Set	Persistent multi-tenant document database

Appendix D: List of papers presented and published

During the tenure of this Capstone Project, the primary focus was directed toward extensive empirical backtesting, stochastic simulation verification, and full-stack cloud and mobile deployment. Publication status:

None. (A technical research paper based on the 38-fold rolling-origin backtesting methodology and Newsvendor inventory optimization results is currently in preparation for submission to an IEEE/SCOPUS-indexed conference).